MACAU UNIVERSITY OF SCIENCE AND TECHNOLOGY

School of Computer Science and Engineering

Faculty of Innovation Engineering

Senior Thesis for the Degree of Bachelor of Science

Title: Unreal Engine based shooter game system design and production

Student Name : SHI WEIHAO

Student No. : 19098533-i011-0057

Supervisor : Professor WONG, Hon-Cheng

August, 2022

澳門科技大學

創新工程學院

計算機科學與工程學院

理學學士學位畢業論文

論文題目: 基于虚幻引擎的射击游戏系统设计与制作

姓　　名 ：时韦豪

學　　號 ：19098533-i011-0057

指導老師 ：黄漢青教授

2022 年 8 月

Abstract

The aim of this thesis is to investigate the development of a battle royale shooter using the Unreal Engine. The popularity of battle royale games has risen dramatically in recent years, making them a popular genre among gamers. The aim of this research is to promote the genre by exploring the potential of the Unreal Engine in the development of Battle Royale shooters.

To achieve this goal, this article goes as far as possible to complete a complete shooter game, to experience the various processes involved in developing a shooter game, and to gain insight into the bottlenecks and difficulties of the development process. This article covers the following main stages: a detailed introduction to the engine used, the design and implementation of the game mechanics and systems, and a summary of the modular game system.

The detailed introduction to using the engine involves a detailed introduction to the engine used, UnrealEngine, and an analysis of the framework, including the Gameplay framework, the blueprint system, C++ programming, the animation system and the UMG section. The Unreal Engine is used to create games and incorporates various features such as a powerful physics engine, dynamic lighting and a responsive user interface. More importantly, the project has detailed named format documentation available for review.

The game system design and implementation phase involved such things as item architecture, backpack item management, route function generation, AI generation and safe zone functionality.

The final part is the modular game system. Why try to use a modular mindset? A complete game system is made up of numerous sub-systems, and as a game programmer, you can't just do the whole game system for the player, but you have to take into account the game plan to change the values to improve the gameplay and so on. This is why it is essential to make a game system that can be adjusted one by one.

Overall, this study demonstrates the potential of the Unreal Engine for developing battle royale shooters. The results of this study can be used to inform future research in this field and contribute to the continued growth and popularity of the genre.

Keywords: battle royale; shooter game; FPS; Unreal Engine; game mechanics; game systems; modular game system;

摘要

这篇文章目的是研究使用虚幻引擎开发一个大逃杀射击游戏。近年来，大逃杀游戏的普及率急剧上升，使其成为游戏玩家中受欢迎的类型。本研究旨在通过探索虚幻引擎在开发大逃杀射击游戏中的潜力来促进这一类型的发展。

为了实现这一目标，本文章尽可能去完成一个完整的射击游戏，去体验开发射击游戏的各个过程，去深入了解开发过程中的瓶颈与困难。本文章主要包括以下几个主要阶段：使用引擎的详细介绍，游戏机制和系统的设计和实现，模块化游戏系统的总结。

使用引擎的详细介绍涉及使用的引擎 UnrealEngine 的详细介绍以及框架分析，包括 Gameplay 框架、蓝图系统，C++编程，动画系统以及 UMG 部分。虚幻引擎被用来创建游戏，并整合了各种功能，如强大的物理引擎、动态照明和响应式的用户界面。更重要的是，这个项目拥有详细的命名格式文档以供查阅。

游戏系统设计和实现阶段涉及如物品架构，背包物品管理，航线功能生成，AI 生成以及安全区功能等等。

最后一部分是模块化游戏系统。为什么要尝试使用模块化的思维呢？一个完整的游戏系统是由无数个子系统去构成，作为一名游戏程序员，不能单单为游戏玩家去完成整个游戏系统，而要考虑到游戏策划去更改数值以提升游戏性等方面。所以去做一个一个可调整的游戏系统是非常必要的。

总的来说，这项研究证明了虚幻引擎在开发大逃杀射击游戏方面的潜力。本研究的结果可以用来为该领域的未来研究提供参考，并为该类型游戏的持续增长和普及做出贡献。

關鍵詞：大逃杀;射击游戏;FPS;虚幻引擎;游戏机制;游戏系统;模块化游戏系统;

Table of Contents

Abstract I

摘要 III

Table of Contents IV

List of Figures VII

List of Tables VIII

Chapter 1 Introduction .....

1.1 Background and motivation .....

1.1.1 The development of the shooting game industry .....

1.1.2 Motivation .....

1.2 Thesis organization .....

Chapter 2 Key technology overview .....

2.1 Unreal Engine 4.27 .....

2.1.1 Unreal Engine introduction .....

2.1.2 Unreal Engine Gameplay Framework .....

2.1.3 Unreal Engine Blueprint System .....

2.1.4 Unreal Engine C++ Programming .....

2.1.5 Unreal Engine Animation System .....

2.1.6 Unreal Motion Graphics .....

2.2 Naming convention of this project .....

2.2.1 Naming convention overview .....

2.2.2 Resource naming convention .....

2.2.3 Folder naming convention .....

2.2.4 Blueprint: Variables, functions and graphs .....

Chapter 3 Requirement analysis and design .....

3.1 Requirements overview .....

3.2 Gameplay Design .....
3.3 3C: Character, Control and Camera Design .....
3.3.1 *Character Design* .....
3.3.2 *Control Design* .....
3.3.3 *Camera Design* .....
3.4 Game item structure .....
3.4.1 *Weapon Type Items* .....
3.4.2 *Equipment Type Items* .....
3.4.3 *Consumable Type Items* .....
3.5 Game interface .....
Chapter 4 Concrete Realization .....
4.1 Character and Interaction System .....
4.1.1 *Character animation blending space* .....
4.1.2 *Character animation aiming space* .....
4.1.3 *Character animation state machines* .....
4.2 Item System .....
4.2.1 *Item architecture* .....
4.2.2 *Random item generation system* .....
4.3 Weapon System .....
4.3.1 *Weapon system data structure* .....
4.3.2 *Weapon functionality system implementation* .....
4.4 AI Spawn System .....
Chapter 5 Conclusion .....
5.1 Conclusion .....
5.2 Future work .....
References .....
Appendix .....

Acknowledgements.....

List of Figures

Figure 2-1 Generic Gameplay Framework .....
Figure 2-2 Specific Gameplay Framework.....
Figure 2-3 Blend Space Framework .....
Figure 3-1 Gameplay flow chart.....
Figure 3-2 Structure of the weapon attachment.....
Figure 3-3 Parachute Jumping Interface.....
Figure 3-4 Userinfo Interface .....
Figure 3-5 Backpack Interface.....
Figure 3-6 Prompts Tips .....
Figure 3-7 Action Tips.....
Figure 4-1 Blend space and animation sequence.....
Figure 4-2 Use Blend Space In Animation Blueprint.....
Figure 4-3 Aimoffset workflow.....
Figure 4-4 Unarmed state machine.....
Figure 4-5 Armed state machine.....
Figure 4-6 Jumping animation state machine .....
Figure 4-7 Skydiving animation state machine .....
Figure 4-8 General item architecture.....
Figure 4-9 Item base class .....
Figure 4-10 Pickup item base class.....
Figure 4-11 Pickup item specify class .....
Figure 4-12 Body item base class.....
Figure 4-13 Item used class .....
Figure 4-14 Item Group Random Generation.....
Figure 4-15 Item Group Random Generation Rules.....
Figure 4-16 Weapon Reload workflow .....
Figure 4-17 Open scope precision aiming .....
Figure 4-18 Random aiming.....
Figure 4-19 AI Spawn System workflow .....

List of Tables

Table 2-1 Common Naming .....
Table 2-2 Animations resource Naming .....
Table 2-3 Artificial Intelligence resource Naming .....
Table 2-4 Blueprint resource Naming .....
Table 2-5 Materials resource Naming .....
Table 2-6 Physics resource Naming .....
Table 2-7 User Interface Naming .....
Table 3-1 Character Type .....
Table 3-2 Equipment Items.....
Table 3-3 Consumed Items .....
Table 4-1 Blend space resource Name .....
Table 4-2 Aimoffset Nane .....
Table 4-3 Animation state machine .....
Table 4-4 Table Weapon Data Structure .....
Table 4-5 Weapon ammo Data Structure .....
Table 4-6 Weapon accessory Data Structure.....
Table 4-7 AI Spawn System Parameter .....

Chapter 1 Introduction

1.1 Background and motivation

1.1.1 The development of the shooting game industry

The world's first first-person shooter (FPS) game, Wolfenstein 3D, was developed by id Software and released in 1992. This groundbreaking title also pioneered the game engine technology, which has since become a crucial component of modern game development [1]. In 1998, Valve Corporation released Half-Life, a game that utilized the GoldSrc engine, a modified version of id Software's Quake engine [2]. The GoldSrc engine facilitated a more immersive gameplay experience and allowed for a smoother plot progression [2]. Additionally, it improved artificial intelligence by enabling more intelligent behavior from in-game characters, such as robots (Leigh, 2014). Counter-Strike, another landmark shooter, was developed as a mod for Half-Life and went on to become a highly popular game in its own right [3].

In recent years, shooter games have grown rapidly as an important branch of the gaming industry. According to Newzoo (2021), the market for shooter games has shown steady growth worldwide and accounts for a significant share of the total revenue of the gaming market.[4]

Shooting games have become one of the main competitive sports in the eSports field due to their high competitive and spectator nature[5]. In recent years, shooting game eSports events have been growing in scale and audience size, giving a huge boost to the shooting game industry[6].

At the same time, shooters have seen significant growth in online multiplayer co-op and competition[7]. Online multiplayer shooters allow players to compete and cooperate in real time on a global scale, enhancing the social aspect of the game and increasing player engagement and loyalty[8].

With the continuous upgrading of game engines, graphics processing technologies and hardware devices, the graphics quality, physical characteristics and interactivity of shooters have been greatly improved[9]. In addition, the application of

virtual reality (VR) and augmented reality (AR) technologies has greatly enhanced the immersion and sense of immersion in shooters[10].

1.1.2 Motivation

The main reason is that I love playing shooters and they have a very strong influence on me, so I had the desire to make a shooter game.

During the development of the shooter game, I was not able to complete the entire game flow very well, but I have completed the basic shooter subsystems and tried to provide a tweakable subsystem to better balance the game later on.

Of course not only that, but from the above background we can clearly see that shooters have a huge potential and growth in the current gaming market. Gaming is one of the valuable cultural manifestations of the human spirit level. It is an emerging cultural industry that lies at the top end of the industry chain in terms of development and so on. With the continuous enrichment of electronic devices and network carriers, the game industry is developing rapidly. The profit margin of a self-developed game product is usually above 50% and its mid-life is relatively long compared to other products, allowing the life cycle of a game product to be extended through continuous improvement[11]. The development of the games industry provides a new form of entertainment for consumers and enhances their experience.

1.2 Thesis organization

Chapter 2, I will give a detailed overview of the development techniques and naming conventions used in my project.

Chapter 3, I will carry out a detailed system requirements analysis and design for the various parts of the game.

Chapter 4, I will implement each game subsystem, the problems encountered during implementation and the solutions.

Chapter 5, I will conclude with some summaries of the project and a description of the work that I will continue to do in the future.

Chapter 2 Key technology overview

This section introduces the game engine tools I will be using, some relevant game development techniques and the naming conventions applied to this project.

2.1 Unreal Engine 4.27

With the rapid development of the gaming industry, various game engines have emerged. Among the many game engines, Unreal Engine 4 (hereinafter referred to as UE4) developed by Epic Games has become the first choice of many developers due to its powerful features and ease of use. UE4 has been successful in many well-known games, such as Fortnite and Escape from Tarkov, PUBG(PlayerUnknown's Battlegrounds). This part will focus on the features of UE4 and how to use UE version 4.27 for Battle Royale shooter development.

2.1.1 Unreal Engine Introduction

Unreal Engine is a real-time 3D computer graphics software engine developed and maintained by Epic Games, an American game company. The core features of Unreal Engine include graphics rendering, physics simulation, audio processing, interaction design, visual scripting and more.[12]

Here are some key features that we may mention in this project:

1. High-performance graphics rendering: Unreal Engine features high-performance graphics rendering capabilities, supporting advanced effects such as real-time ray tracing, global lighting, physics simulation and more. With its powerful rendering technology, developers can achieve realistic graphics effects and improve the visual performance of their projects.
2. Powerful Blueprint System: Unreal Engine's Blueprint System is a node-based visual programming tool that allows developers to quickly build game logic, AI behaviour, interaction design and other features without writing code. The Blueprint System lowers the bar for programming, making it easy for non-programmers to participate in project development.
3. Character and animation system: UE4.27 offers a powerful character creation and animation system. Developers can create unique in-game characters for

Battle Royale shooters with the help of preset models and animation resources, or custom models and animations. Also, with UE4's skeletal animation system and animation blueprints, character movements such as moving, shooting and jumping can be easily realised, giving a vivid behavioural representation to the game characters.

4. Visual editors: The UMG provides a set of visual editors that allow developers to quickly create and modify UI elements by dragging and dropping components, adjusting properties and more, without writing code.

2.1.2 Unreal Engine Gameplay Framework

The Gameplay Framework of Unreal Engine provides a powerful set of classes to build my game. In this game, the framework is very flexible and does some heavy lifting and sets some standards. The following diagram shows the generic game framework.

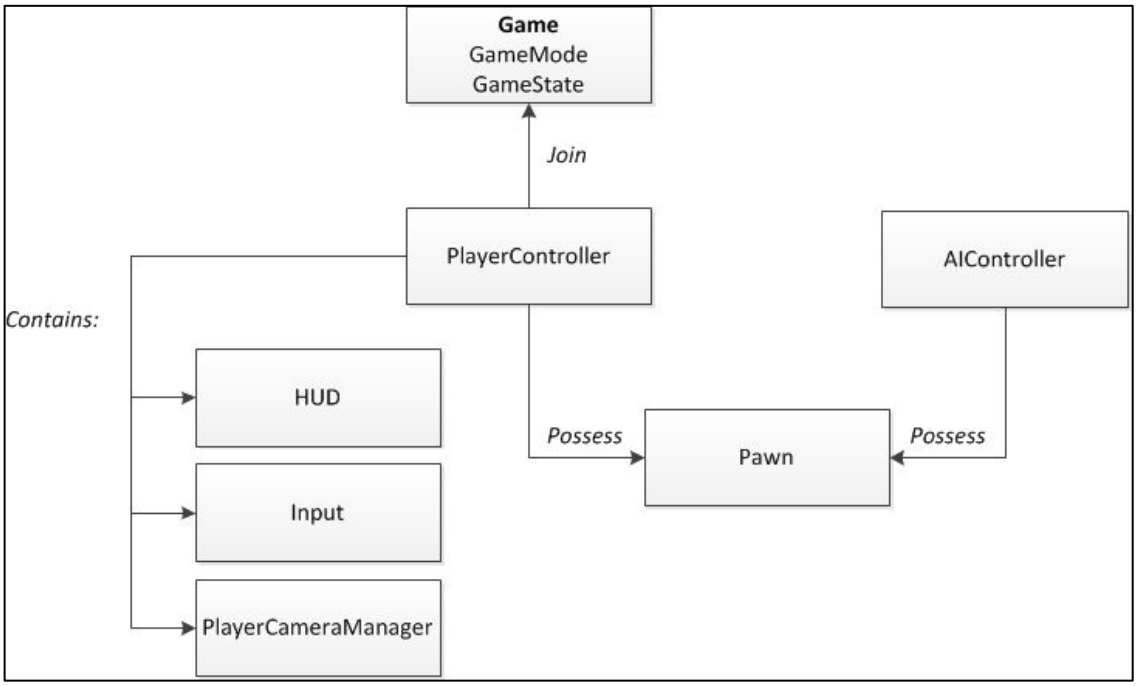


Figure 2-1 Generic Gameplay Framework

The following diagram shows this project gameplay framework.

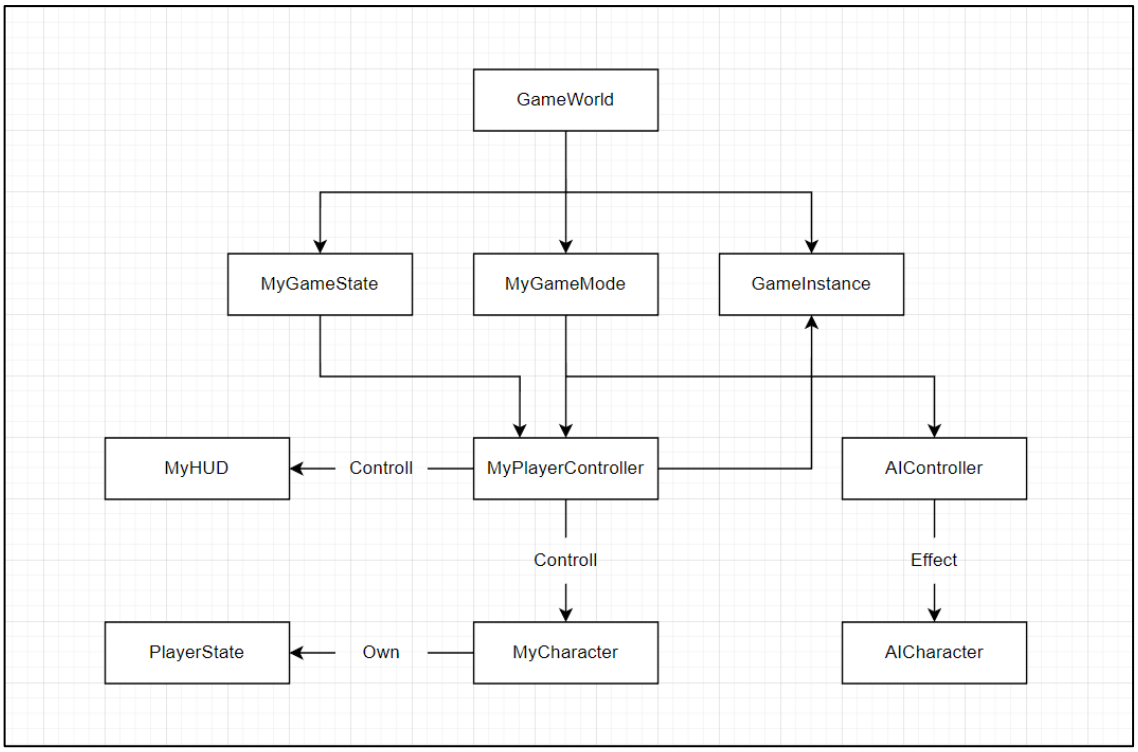


Figure 2-2 Specific Gameplay Framework

MyGameMode: In Unreal Engine, Game Mode is a key game framework component that defines the core rules and logic of the game. Game Mode is responsible for the following: the basic rules and constraints of the game, assigning player controllers and characters, controlling session flow, levels and scenarios, etc. For this project, Gamemode plays a role in randomly generating aircraft routes, randomly generating the number of items, implementing the poison circle mechanism, etc.

MyGameState: In Unreal Engine, Game State is an important game framework component used to store and manage information that persists throughout a game session. Game State is shared by all players and is typically used to synchronise global information in multiplayer games. The main functions of Game State include: recording global data, managing lists of player states, synchronising multiplayer information and more. Implemented into this project are items such as recording the number of survivors remaining in the game, time remaining, etc. In Unreal Engine, game state is a Blueprint class or C++ class, usually specified in the project settings or level settings along with

Game Mode. The Game State class usually contains events and functions to handle game logic and data updates.[13]

GameInstance: In Unreal Engine, a Game Instance is an important game framework component whose primary purpose is to store and manage persistent data throughout the life of a game application. A Game Instance is created when the game is started and destroyed when the game is closed, thus maintaining a persistent presence between levels and scenarios of the game.[14] Its main functions are to store persistent data, manage global resources, implement cross-level logic, etc. Implementation of this item is such as recording the names of fixed character or weapon skeleton slots, specific damage to guns, specific capacity of backpacks, weapon information, etc.

MyHUD: In Unreal Engine, the HUD (Heads-Up Display) is a key game framework component used to draw and display game interface elements such as blood bars, scoreboards, maps, hint messages, etc. on the screen. the HUD allows the player to know key information and status in real time during gameplay[15]. It is not just one specific UI interface, but a management component that controls and manages all UI interfaces. For example, when to draw the main game screen, when to draw the backpack screen, when to respond to player input, etc.

Myplaycontroller&MyCharacter: Here, I have chosen to explain them together, the two framework components are very strongly related. The Player Controller is the component that is used to receive input from the player at this point in time, and the Mycharacter component is like the Muppet in that it is influenced by the input from the Player Controller. Our player controller does not only control mycharacter, but can also perform other interactive actions such as driving a car, flying a plane, etc.

In Unreal Engine, Player State is an important game framework component for storing and managing information related to a specific player. The main functions of Player State include: recording player data, managing player information and synchronising multiplayer information, among others[16]. The implementation of this project represents functions such as the storage of backpack items, current character kills, current character health, and other character-specific information about the game client.

2.1.3 Unreal Engine Blueprint System

Unreal Engine's Blueprint system is a visual script-based programming method that allows developers to create and edit game logic without writing actual code. The Blueprint system allows designers and programmers to collaborate more effectively, speeding up the game development process. The Blueprint system is particularly powerful for implementing prototypes, so if I have an idea for gameplay, I can prototype it very quickly and start iterating. The first iterations of this project were built using the blueprint system, with subsequent rewrites in C++ for some of the performance-hungry parts.

While the Unreal Engine's blueprint system brings many conveniences to game development, it also has a number of flaws:

1. Performance issues: Probably the biggest issue with the implementation is the potential performance gap in the blueprint system compared to code written in C++[17]. As blueprints are run on a virtual machine, they are typically slower to execute than native C++ code.
2. Complexity management: When projects are large or blueprint logic is complex, blueprint systems can become difficult to maintain. Too many nodes and connecting lines can make the logic difficult to understand and make debugging and optimisation more difficult. This shortcoming is evident in this project, where the Myplayercontroller has a very large input receiving function, which makes the connection lines very complex and difficult to read.
3. Code reuse and modularity: While the Blueprint system supports the creation of functions and custom events to improve code reuse, it still has limitations in terms of code modularity and reuse compared to C++. This shortcoming is not evident in this project.

2.1.4 Unreal Engine C++ Programming

Unreal C++ programming is the process of writing game logic and engine extensions in C++ within the Unreal Engine, which provides developers with a powerful set of C++ programming interfaces that allow programmers to take full advantage of the

performance benefits of C++ while taking advantage of the highly integrated nature of the engine to develop games quickly[18].

Unreal C++ programming has the following features:

1. High performance: C++ code typically executes faster than virtual machine-based blueprint systems, making it particularly suitable for performance-critical gaming projects.
2. Powerful extensibility: Unreal C++ programming allows developers to access the underlying system directly, providing more customisation options for game and engine functionality.
3. Highly integrated: Unreal C++ programming is tightly integrated with other modules of the engine, such as graphics rendering, physics simulation, animation systems, etc., enabling cross-functional game logic

2.1.5 Unreal Engine Animation System

The animation system in Unreal Engine 4 (UE4) is made up of several animation tools and editors that mix skeleton-based mesh deformation with morph-based vertex deformation to allow for complex animations. This system is the one I focused on learning in this project. This system can be used to make the player's basic movements appear more realistic by playing and blending canned animation sequences, create custom special movements such as using animation montages to control the repetitive movements of an animation sequence, using skeletal controls to directly control the transformation of bones, or creating logic-based state machines to determine which animation a character should use in a given situation[19].

Below I will focus on the use of the Unreal Engine subsystem features that I used in this project:

1. Blended animation space: The blend space is a special resource that can be sampled in the animation diagram, allowing animation to be blended based on the values of two inputs. To achieve a simple blend between two animations based on one input, a standard blend node is provided in the animation blueprint. The blend space provides the means to perform more complex

blending between multiple animations based on multiple values (currently limited to two).

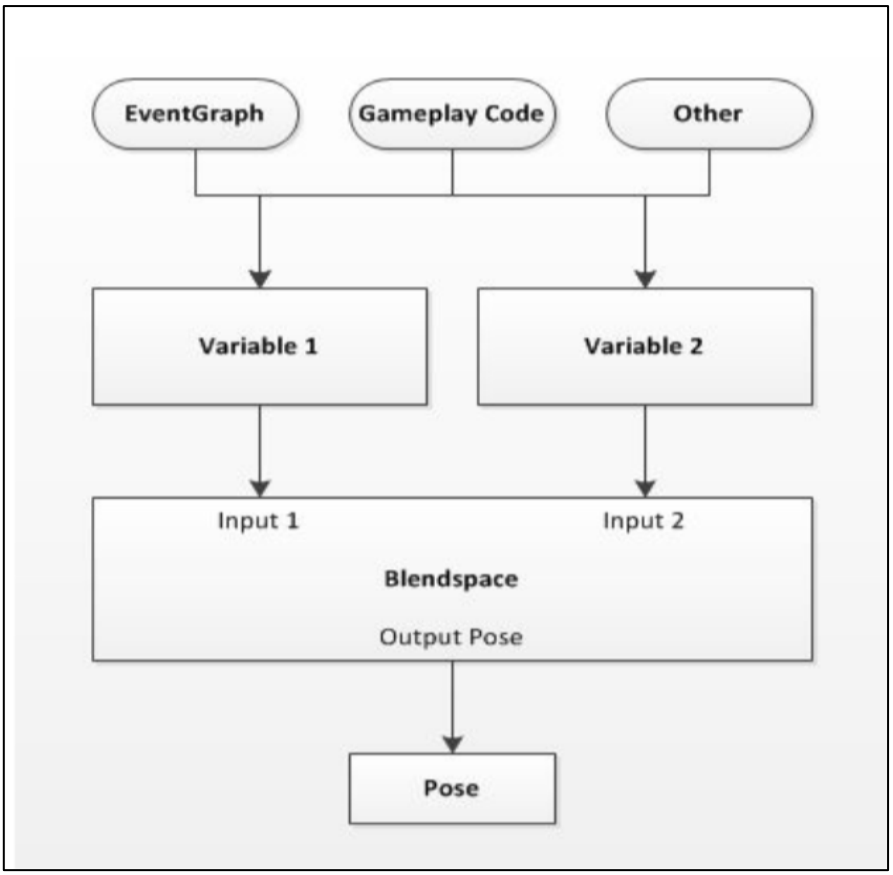


Figure 2-3 Blend Space Framework

2. Animated aiming offsets: Aiming offsets are a resource that stores a series of blendable poses to help the character aim a weapon. During animation, the results of aiming offsets are blended with other movements (such as running, walking, jumping etc.) to make the character look smooth from all directions.
3. Animation blueprints in Unreal Engine are special blueprints that handle the animation logic of game characters and other animatable objects. Animation blueprints allow developers to create and control a character's animation state machine, handle animation events, and interact with other objects in the game world in a visual environment. The animation blueprint consists of two main components:

   The animation diagram is responsible for defining the logical flow of animation, including blending, layering, state switching, etc.

The animation state machine is a tool used to manage the state of the character's animations and can control the character's animation play according to the game logic.

2.1.6 Unreal Motion Graphics

Unreal Motion Graphics (UMG) is a UI design and implementation system for the Unreal Engine that allows developers to create game interfaces using a drag-and-drop visual editor, including various controls (such as buttons, text boxes, scrollbars, etc.), and to add interaction logic and animations to these controls using a blueprint system.

Its main features are as follows:

1. Visual editor: UMG provides an intuitive editor in which developers can create and design UI elements, adjust layout and style, without writing code.
2. Blueprint integration: UMG is tightly integrated with the Unreal Engine's blueprint system, allowing developers to write interaction logic and animation effects for UI elements that interact with other objects in the game world.
3. Rich UI controls: UMG provides many predefined UI controls such as buttons, textboxes, listboxes, sliders, etc. for developers to quickly build game interfaces.
4. Responsive layout: UMG supports responsive layouts, making it easy for developers to create game interfaces that adapt to different screen sizes and resolutions.
5. Componentisation: UMG supports the creation of reusable UI components (User Widgets), which help to organise and manage the complexity of the game interface.

2.2 Naming convention of this project

Clear and consistent naming conventions make it easier for us to understand and work with code, especially in large projects. They also make it easier to identify reusable code and promote code extensibility. Following a unified naming convention reduces communication costs and misunderstandings among team members, facilitating code collaboration and merging. Adhering to a UE4 game project naming conventions

is crucial for the development and maintenance of large-scale projects, as it can enhance development efficiency, code quality, and reduce errors and communication costs.

Here are the details of my project naming convention.

2.2.1 Naming convention overview

Unfortunately, I didn't realize the importance of naming conventions when I first tried to make this game. But at a later stage, I added the naming convention in time.

There are many different naming conventions on the Internet, and different people and different teams use different naming conventions. I sifted through the Internet and found a set of naming conventions that many people use and agree with. This naming convention is the Gamemakin UE4 Style Guide. It’s heavily inspired by the Airbnb Javascript Style Guide.

I will describe the naming convention of this project in detail below.

2.2.2 Resource naming convention

Most basic resource names should follow this form Prefix_BaseAssetName_Variant_Suffix. Naming conventions should be treated as law. A project that conforms to a naming convention is able to have its assets managed, searched, parsed, and maintained with incredible ease.

Prefix and Suffix are to be determined by the asset type through the following Asset Name Modifier tables. BaseAssetName should be determined by a short and easily recognizable name related to the context of this group of assets. For example, I had a character named MyCharacter, all of MyCharacter 's assets would have the BaseAssetName of MyCharacter. For unique and specific variations of assets, Variant is either a short and easily recognizable name that represents logical grouping of assets that are a subset of an asset's base name. For example, if MyCharacter had multiple skins these skins should still use MyCharacter as the BaseAssetName but include a recognizable Variant. An 'Evil' skin would be referred to as MyCharacter_Evil and a 'Retro' skin would be referred to as MyCharacter _Retro.

About the Prefix and Suffix, Below are the tables of all the resource modifications that might be used.

a. Most common Prefix and Suffix to use in the project, as shown in Table 2-1.

Asset Type	Prefix	Suffix
Level / Map		
Level (Gameplay)		_Gameplay
Blueprint	BP_	
Material	M_	
Static Mesh	S_	
Skeletal Mesh	SK_	
Texture	T_	
Particle System	PS_	
Widget Blueprint	WBP_	

Table 2-1 Common Naming

b. The Animations resource Prefix and Suffix to use in the project, as shown in Table 2-2.

Asset Type	Prefix	Suffix
Aim Offset	AO_	
Animation Blueprint	ABP_	
Animation Composite	AC_	
Animation Montage	AM_	
Animation Sequence	A_	
Blend Space	BS_	

Skeletal Mesh	SK_	
Skeleton	SKEL_	

Table 2-2 Animations resource Naming

c. The Artificial Intelligence resource Prefix and Suffix to use in the project, as shown in Table 2-3.

Asset Type	Prefix	Suffix
AI Controller	AIC_	
Behavior Tree	BT_	
Blackboard	BB_	
Decorator	BTDecorator_	
Service	BTService_	
Task	BTTask_	
Environment Query	EQS_	
EnvQueryContext	EQS_	Context

Table 2-3 Artificial Intelligence resource Naming

d. The Blueprint resource Prefix and Suffix to use in the project, as shown in Table 2-4.

Asset Type	Prefix	Suffix
Blueprint	BP_	
Blueprint Component	BP_	Component
Blueprint Function Library	BPFL_	

Blueprint Interface	BPI_	
Blueprint Macro Library	BPML_	
Enumeration	E	
Structure	F or S	
Tutorial Blueprint	TBP_	
Widget Blueprint	WBP_	

Table 2-4 Blueprint resource Naming

e. The Materials resource Prefix and Suffix to use in the project, as shown in Table 2-5.

Asset Type	Prefix	Suffix
Material	M_	
Material (Post Process)	PP_	Component
Material Function	MF_	
Material Instance	MI_	
Material Parameter Collection	MPC_	
Subsurface Profile	SP_	
Physical Materials	PM_	
Decal	M_, MI_	_Decal

Table 2-5 Materials resource Naming

f. The Physics resource Prefix and Suffix to use in the project, as shown in Table 2-6.

Asset Type	Prefix	Suffix
Physical Material	PM_	
Physics Asset	PHYS_	
Destructible Mesh	DM_	

Table 2-6 Physics resource Naming

g. The User Interface resource Prefix and Suffix to use in the project, as shown in Table 2-7.

Asset Type	Prefix	Suffix
Font	Font_	
Slate Brush	Brush_	
Slate Widget Style	Style_	
Widget Blueprint	WBP_	

Table 2-7 User Interface Naming

2.2.3 Folder naming convention

Equally important as asset names, the directory structure style of a project should be considered law. Asset naming conventions and content directory structure go hand in hand, and a violation of either causes unneeded chaos.

In this style, I will be using a structure that relies more on filtering and search abilities of the Content Browser for those working with assets to find assets of a specific type instead of another common structure that groups asset types with folders.

Here is a summary of all the style rules:

a. Use PascalCase:

   PascalCase refers to starting a name with a capital letter and then instead of using spaces, every following word also starts with a capital letter.

b. Never Use Spaces

   Spaces can cause various engineering tools and batch processes to fail.

c. Use A Top Level Folder For Project Specific Assets

   All of a project's assets should exist in a folder named after the project. For example, if your project is named 'Generic Shooter', all of it's content should exist in Content/ProjectName/AllFolder.

   Reason:

   1. No Global Assets: Often in code style guides it is written that you should not pollute the global namespace and this follows the same principle. When assets are allowed to exist outside of a project folder, it often becomes much harder to enforce a strict structure layout as assets not in a folder encourages the bad behavior of not having to organize assets. Every asset should have a purpose, otherwise it does not belong in a project. If an asset is an experimental test and shouldn't be used by the project it should be put in a Developer folder.
   2. Reduce Migration Conflicts: In one project, if we want to copy some resources into the project, the easiest way to perform the copy is to use the Content Browser's Migrate functionality as it will copy over not just the selected asset but all of its dependencies. These dependencies are what can easily get you into trouble. If two project's assets do not have a top level folder and they happen to have similarly named or already previously migrated assets, a new migration can accidentally wipe any changes to the existing assets. This is also the primary reason why Epic's Marketplace staff enforces the same policy for submitted assets.

d. Use Developers Folder For Local Testing

   The role of the developer folder is to provide a convenient workspace for developers to easily access and manage all the necessary files and tools. In the Developer Folder, developers can edit and test game code, import and work

with game material, configure game engines and tools, create and manage game data, and view and write documentation and instructions.

In addition to providing a convenient working environment, the Developer Folder also helps developers to organise and manage game projects to better control and coordinate game development progress. By using the version control system in the developer folder, developers can track and manage all changes to code and material, avoiding version conflicts and data loss.

e. All maps file should be belong in a folder called Maps.
Map files are incredibly special and it is common for every project to have its own map naming system, especially if they work with sub-levels or streaming levels. No matter what system of map organization is in place for the specific project, all levels should belong in /Content/Project/Maps.
Being able to tell someone to open a specific map without having to explain where it is a great time saver and general 'quality of life' improvement. It is common for levels to be within sub-folders of Maps, such as Maps/Campaign1/ or Maps/Arenas, but the most important thing here is that they all exist within /Content/Project/Maps.
This also simplifies the job of cooking for engineers. Wrangling levels for a build process can be extremely frustrating if they have to dig through arbitrary folders for them. If a team's maps are all in one place, it is much harder to accidentally not cook a map in a build. It also simplifies lighting build scripts as well as QA processes.

2.2.4 Blueprint: Variables, functions and graphs

This section will focus on Blueprint classes and their internals. When possible, style rules conform to Epic's Coding Standard.

Variables:

1. All variable names must be clear, unambiguous, and descriptive nouns. And these variables should be in the form of PascalCase.

2. The naming of variables in the blueprint needs to take into account the contextual environment to avoid repeating unnecessary definitions.
3. Do not include native variable type names in variables: Native variables are the simplest types of variables that hold data, such as booleans, integers, floating point numbers and enumerations.
4. Try not to make a variable life a private type unless the variable was intended to always be accessed internally by the class in the first place and the class itself was not intended to be inherited. Try to use protected. The private type is used when you have a very clear reason to restrict the ability of a subclass.
5. Arrays and collections: For variables of the array and collection types, names in the plural form can be used to indicate that they contain multiple elements. For example, an array containing more than one enemy could be named "enemies"

Functions and Graphs: This section is used to explain how functions, events and event dispatchers should be managed. Unless otherwise stated, all the rules that apply to functions also apply to events

1. All functions and event executors are required to do some action, perhaps to fetch information, or to compute data, or to do something. Therefore, all functions should start with a verb and be in the general modern tense, and have a context to indicate exactly what they are doing.
2. If a function does not change the state of the class and only returns information, state or calculations back to the caller yes/no, this should be an interrogation function.
3. Both the response function and the dispatch function of an event should start with On and then follow the verb rule, if it is in the past tense then the verb should be moved to the end for easier reading. When following the verb rule, the English fixed form needs to be given priority.
4. Any time an RPC function is created, it should be prefixed with the target as the prefix, e.g. Server, Client or Multicast, no exceptions. The part after the prefix follows the other rules above.
5. All functions should have return nodes: The return node clearly marks the end of the blueprint's execution at this point. The structure in the blueprint may consist of a parallel structure Sequence, a loop structure ForLoopWithBreak

or a reverse return node. The clear end node makes the blueprint easy to read maintain and debug.

Chapter 3 Requirement analysis and design

When making any software or a game, we have to plan and think about what features are needed for the project and how to design and make that feature before we make it. This is quite important for game development and this chapter will provide a systematic analysis of the requirements for this game.

3.1 Requirements overview

The game was initially conceived as a survival shooter, but due to personal technical constraints and time constraints, it was not possible to fully build the game from all angles. However, during the development process it has been possible to complete the various subsystems and features of the game well, and to make dynamic adjustments at a later stage. Below I will explain my requirements analysis process in the development process.

The basic setting of the game should be determined first, including the game setting, map type, weapon system, etc[21]. This will help the design team to build engaging virtual worlds and provide a unique gaming experience for players[22].

Character design is one of the core elements of a shooter game and needs to provide the player with a customisable in-game character[23], including gender, appearance, clothing, etc. At the same time, the movement design should include basic character movements (e.g. walking, running, jumping, etc.) and combat movements (e.g. shooting, fighting, defending, etc.).

Battle royale shooters often have a variety of weapons and props. The requirements analysis should include the type of weapon, range, type of ammunition, etc. In addition, the game's prop system needs to be considered, such as medical kits, armour, traps, etc.[22]

Interface and interaction design is key to ensuring that players can easily get to grips with the game[24]. The requirements analysis should cover the main interface, in-game menus, setting options, prompt messages, etc. It is also important to consider in-game communication methods, such as voice chat and text chat.

3.2 Gameplay Design

On a randomly generated island, the player will be on a plane generated by a random route. The player will have to perform a parachute jump within a certain distance and may encounter a random number of random types of AI generated by the AI generation system to collect supplies and obtain exit information within a certain time frame. Below is a simple diagram of how the game is played.

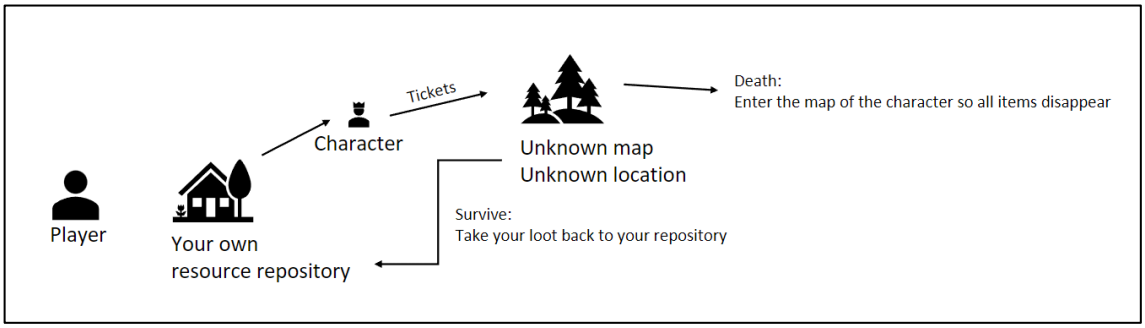


Figure 3-1 Gameplay flow chart

3.3 3C: Character, Control and Camera Design

In game development, the 3Cs refer to the three core components of Character, Camera and Control. They are fundamental elements of game design and are essential to the playability, immersion and user experience of a game.

3.3.1 Character Design

Character orientation: According to the game player type theory proposed by Bartle[25], game characters should cater for multiple player types, including achievers, explorers, socialisers and killers. To this end, we can design a variety of professional characters, such as snipers, assaulters, and medics, each with unique skills and abilities. Considering that this project is a single player type of game, the characterisation aspect is mainly aimed at the individual survival type rather than the cooperative type. Therefore, I have come up with the following general character orientations:

Character type	Description

Explorers	Have a higher recovery rate than other characters when collecting supplies
Survivors	Have a higher starting blood count than others and have a higher physical immunity to damage
Medics	Self-possessed therapeutic function
killers	Higher damage to the enemy

Table 3-1

Character Development: Character growth is one of the core drivers of games. Game characters should have some growth, including upgrades, unlocking new skills, and changing their appearance. This will add challenge and depth to the game, while satisfying the player's need for achievement[26].

Character animation fluency: Self-character animation very much affects the feel and smoothness of the game's controls.

3.3.2 Control Design

Simple controls: ease of use is an important factor in game playability. The basic controls of the game should be simple to understand and logical for most gamers to use. It should be easy for novice players to get used to the game quickly, e.g. WASD keys to control character movement, left mouse button to shoot, right mouse button to aim, F key to interact, etc.

Good feedback enhances the player's sense of action and improves the playability of the game[27]. Feedback in games can include special effects when hitting enemies, sound cues, etc.

3.3.3 Camera Design

Perspective choice: The perspective of the game directly affects the player's gaming experience[28]. Games should offer a variety of perspectives for players to choose from, such as first-person perspective (FPS) and third-person perspective (TPS).

This project focuses on the TPS aspect of the design, with the FPS aspect only being enabled when shooting and aiming.

Camera Shake: Camera shake is a very important part of the camera movement simulation, it directly affects or tells the player the status of the character at the moment and gives the player a strong sense of immersion and experience. For example, when running, the camera will shift slightly up and down or left and right to simulate camera motion turntables. When hit, the camera shakes violently and quickly to represent a feedback of the hit.

3.4 Game item structure

A successful battle royale shooter game requires a rich and varied item system to enhance the playability and appeal of the game. When designing the item architecture, the balance, strategy and ease of use of the game, the collectability of the items and other factors need to be fully considered.

When designing the item system, it is important not only to be able to design the item system, but also to expose the parameters for the system, such as the values, so that the game plan can be adjusted in a targeted manner.

3.4.1 Weapon Type Items

Weapons are one of the core elements of a shooter. A rich weapon system provides players with a variety of combat styles, enhancing the strategy and challenge of the game[29].

Types of weapon: These include pistols, submachine guns, shotguns, rifles and sniper rifles, each with different attributes such as range, rate of fire and damage. However, only the rifle and sniper types have been implemented in this project, but weapons will continue to be added later.

Weapon accessories: such as scopes, silencers, diffusers, etc., can improve the performance of a firearm or provide the player with additional tactical options. Below is a chart of the weapon gun accessories for this item.

Figure 3-2 Structure of the weapon attachment

Bullet type items: Different types of weapons will use different types of bullets, and different bullets will take up different amounts of space.

3.4.2 Equipment Type Items

Armor can enhance a player's survivability and strengthen a game's replayability[23]. Defence items are key to improving a player's ability to survive. Choosing the right combination of defence gear will allow players to better cope with various combat situations and increase their win rate in the game. Players need to keep searching for higher level of defence gear during the game to ensure a stronger resistance in case of enemy encounters.

Backpacks do not provide direct protection, but they can increase a player's ability to carry items, allowing them to carry more defence, weapons and props. Backpacks are usually divided into different levels, with the higher the level, the more space they provide.

In-game skins are a virtual commodity which is also an important source of income in commercial games. It offers the player a certain aesthetic effect and is an expression of social identity.

Here is the table of equipment items:

Name	Level
Helmet	1
Helmet	2
Helmet	3
Vest	1
Vest	2
Vest	3
Backpack	1
Backpack	2
Backpack	3
Fashion clothing	No Level

Table 3-2

3.4.3 Consumable type items

Consumables play a key role in battle royale shooters. They can provide the player with lifesteal restoration, enhanced attributes and strategic adjustments that enhance the fun and depth of the game. It will generally serve several purposes:

1. Vitality recovery: consumables are often used as the primary means of vitality recovery in the game [30]. For example, first aid kits and bandages can quickly restore a percentage of lifesteal and gradually restore a small amount of lifesteal, respectively. Through lifesteal recovery, consumables help to increase the player's survival and persistence in the game.
2. Enhanced attributes: Certain consumables can provide temporary attribute gains [30]. For example, energy drinks and painkillers can increase a player's movement speed, aim stability, and progressively regain vitality over a period

of time These attribute gains provide players with more tactical options and combat advantages [31].

3. Strategic adjustments: consumables can also influence players' strategic and tactical choices. For example, by allocating and using consumables wisely, players can strategically adjust to different opponents and battle situations to achieve higher game outcomes.

Here is a table of the types of items consumed:

Name	Treatment level
Medical kit	3
First Aid Kit	2
Bandages	1
Adrenaline	3
Painkillers	2
Energy drinks	1

Table 3-3

3.5 Game interface

The Game User Interface (GUI) is the key medium through which players interact with the game, conveying information to the player and receiving input from the player. An efficient, intuitive and aesthetically pleasing game interface can greatly enhance the player's gaming experience.

Main Menu: The main menu is the entry point to the game interface and usually includes the game start, setup options, help documentation, etc. The main menu should allow the player to easily access the game and perform various actions.

Player jump screen: the screen used when the player falls from the aircraft to the ground, in order to better inform the player of information such as the current speed of descent and altitude.

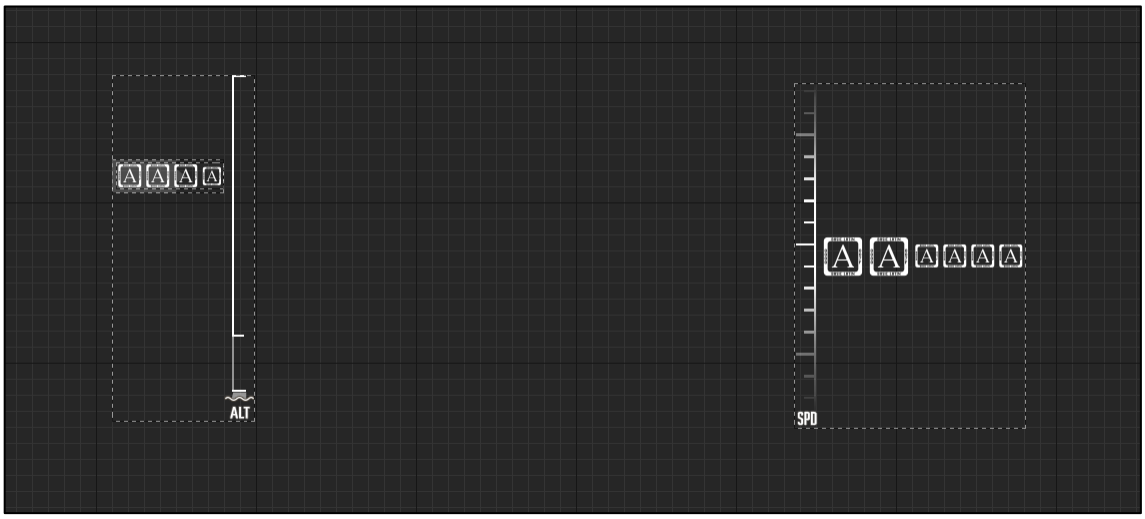


Figure 3-3 Parachute Jumping Interface

Player information display: display real-time information on the game screen, such as character life, ammunition, map, etc. The HUD should be simple and easy to read so that players can quickly access key information during the game.

Figure 3-4 Userinfo Interface

Player backpack screen: The player backpack screen represents the items and information in the player's backpack at the moment, allowing the player to better plan the contents of their backpack.

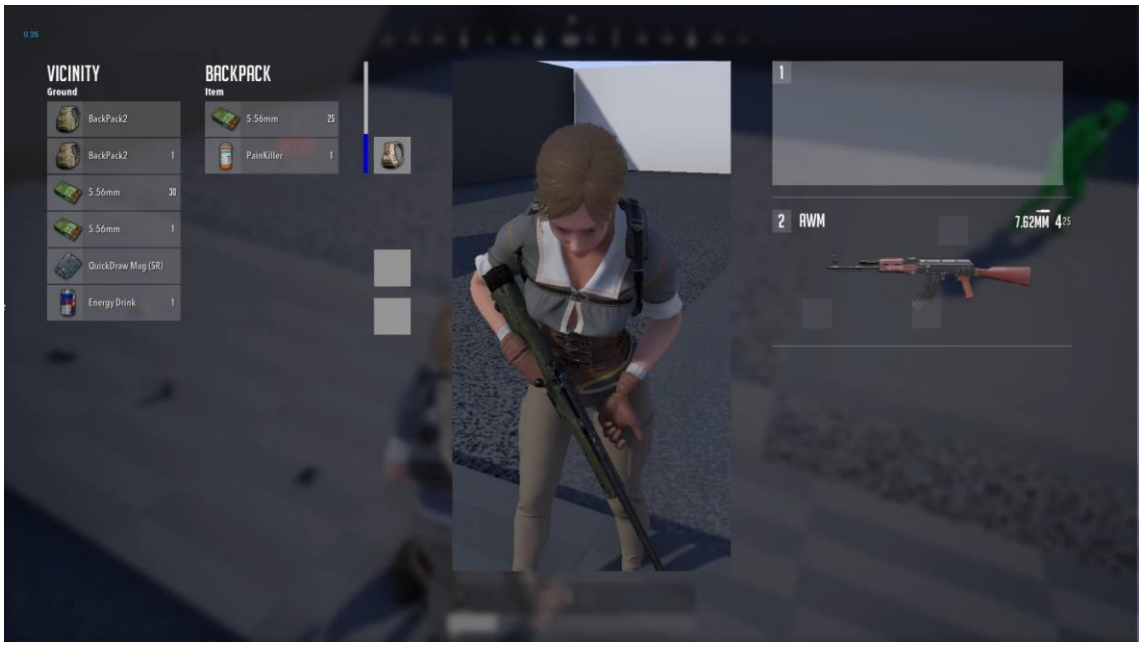


Figure 3-5 Backpack Interface

Dialogs and Prompts Interface: Dialogs and prompts are used to communicate the game story, mission information or action tips to the player. They should be brief and concise, avoiding length and complexity, so that the player can quickly understand them and make decisions accordingly. For example, the message "Press F to pick up" when you encounter an item that can be picked up, or "Backpack is full" when it is full.

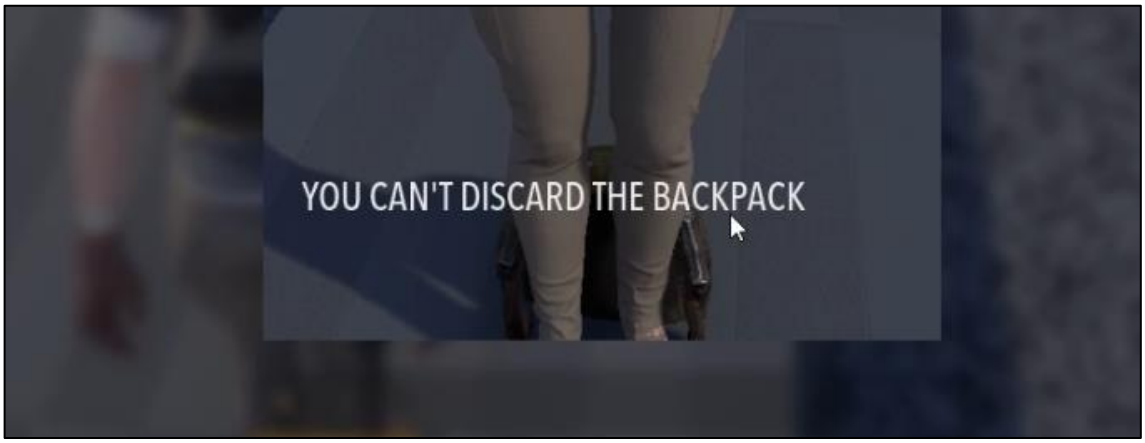


Figure 3-6 Prompts Tips

Figure 3-7 Action Tips

Chapter 4 Concrete Realization

This section focuses on the implementation of the key subsystems of the project and describes the major problems encountered during the development process and how they were resolved.

4.1 Character and Interaction System

This section focuses on the basic character movement and interaction system.

4.1.1 Character animation blending space

Blend Space reduces the need to create individual hard-coded nodes for blending animations by using resources that blend based on specific properties or conditions. It enables the animator or programmer to specify the input, the animation, and the way in which the input is blended between animations, and virtually any type of blending can be performed through the blend space generic.

The picture below shows how the blend space is used with the produced animation sequence.

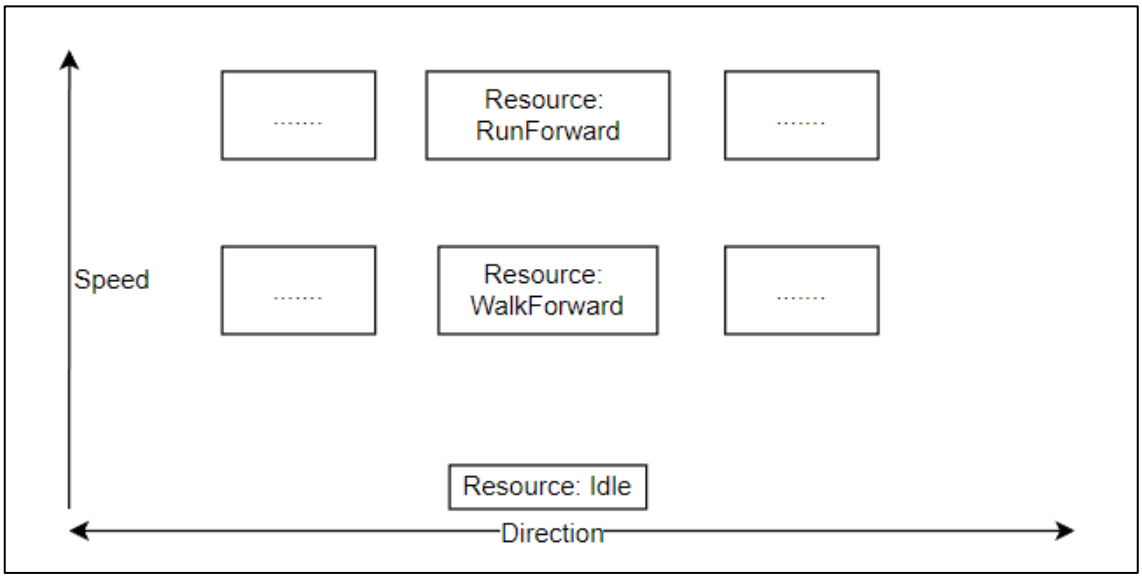


Figure 4-1 Blend space and animation sequence.

Now that we have a blend space that can be adjusted with input, the following figure explains how the animation blend space is used in this project.

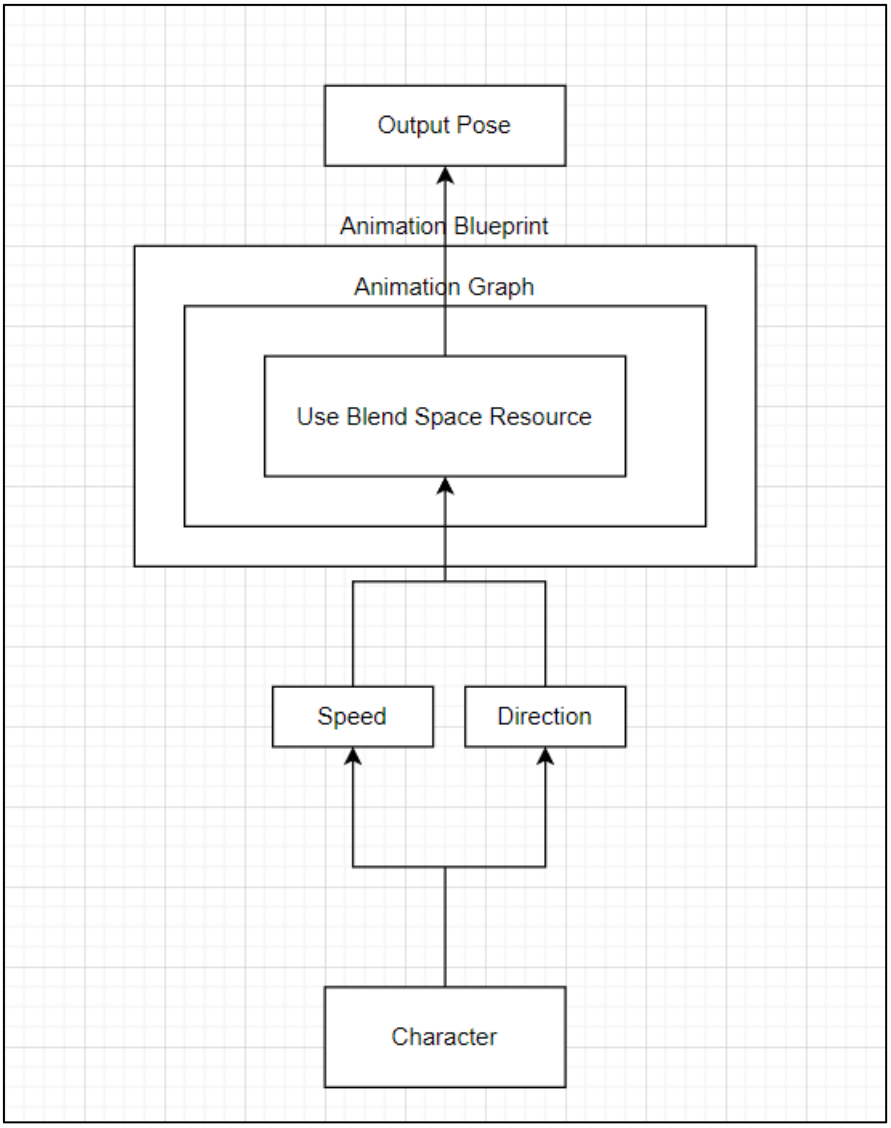


Figure 4-2 Use Blend Space In Animation Blueprint

The table below is the blend space resource table created by this project:

Blend Space Name
Non-gun status - Stand
Non-gun status - Crouch
Non-gun status - Prone

Gun-status - Stand
Gun-status - Crouch
Gun-status - Prones

Table 4-1

4.1.2 Character animation aiming space

Aim Offset in Unreal Engine is a technique for creating smooth aiming animations. It allows characters to naturally adjust their body pose when aiming at different targets without having to create separate animations for each possible aiming direction. This technique is mainly used in third and first person shooters, but can also be used in other game types that require aiming.

The basic principle of Aim Offset is to take the character's aiming direction (e.g. horizontal and vertical angles) as an input parameter and interpolate it between pre-made animation clips to generate the character's natural pose when aiming at a target. Typically, these animation clips include aiming animations with the character facing different directions (e.g. aiming up, down, left and right).

To create an Aim Offset in Unreal Engine, you will need to complete the following steps:

1. First, create a set of aiming animations for your character, including aiming animations facing different directions.
2. Create a new Aim Offset asset. This is a special type of Blendspace used to interpolate between pre-made animations based on the input parameters (aiming angle).
3. Add the previously created aiming animations to the Aim Offset asset and set the corresponding aiming angle for each animation. This will create a 2D grid for interpolating animations at runtime based on the character's aiming direction.

4. In the character's animation blueprint, add the Aim Offset node to the state machine and pass the character's aiming direction as an input parameter to the Aim Offset node. This will ensure that the appropriate animation is automatically calculated and played at runtime based on the character's aiming direction.

The table below is the aimoffset resource table created by this project:

Aimoffset Name
AimOffset_NormalStand
AimOffset_NormalCrouch
AimOffset_RifleAimCrouch
AimOffset_RifleCrouch
AimOffset_RifleStand
AimOffset_RifleAimStand

Table 4-2

The following diagram shows how character input affects aimoffset:

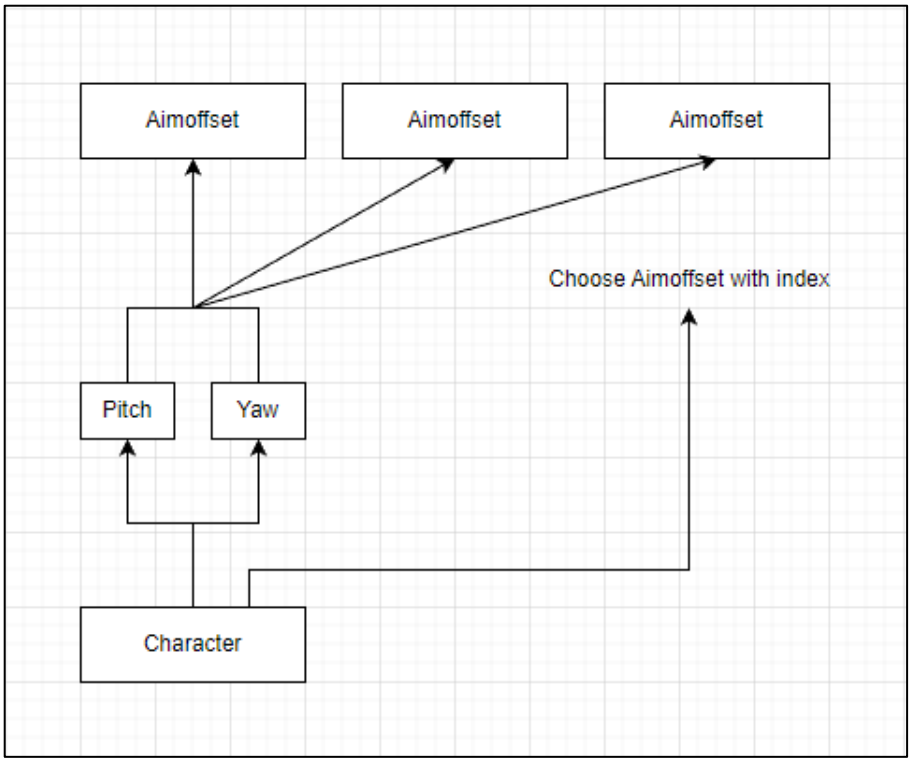


Figure 4-3 Aimoffset workflow

4.1.3 Character animation state machine

The picture below shows the non-gun holding status machine:

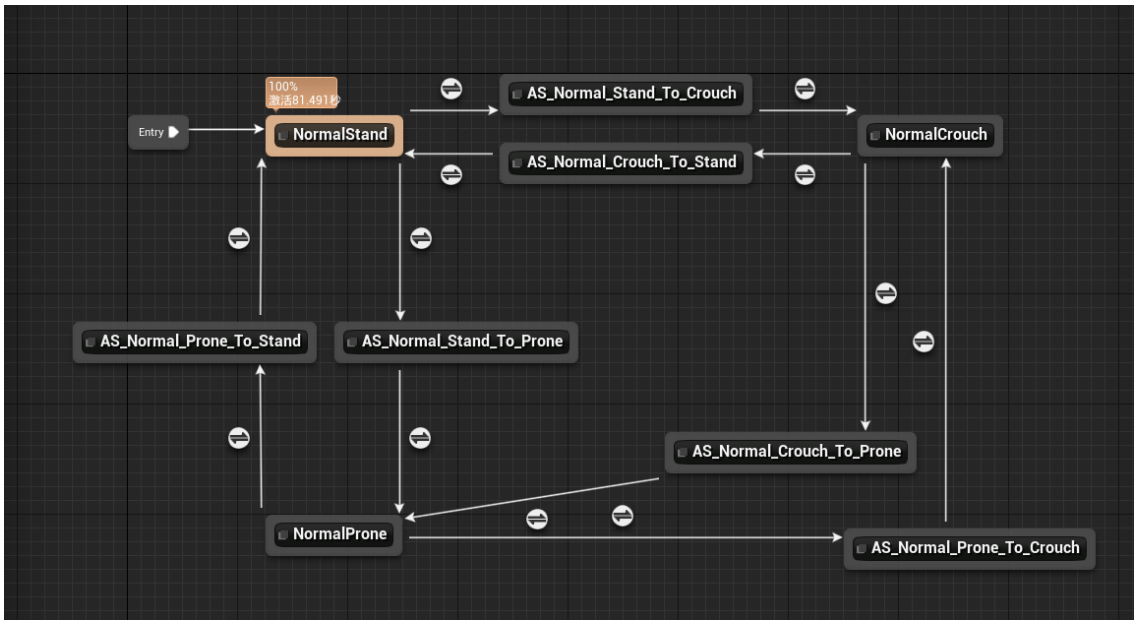


Figure 4-4 Unarmed state machine

The picture below shows the gun holding animation state machine:

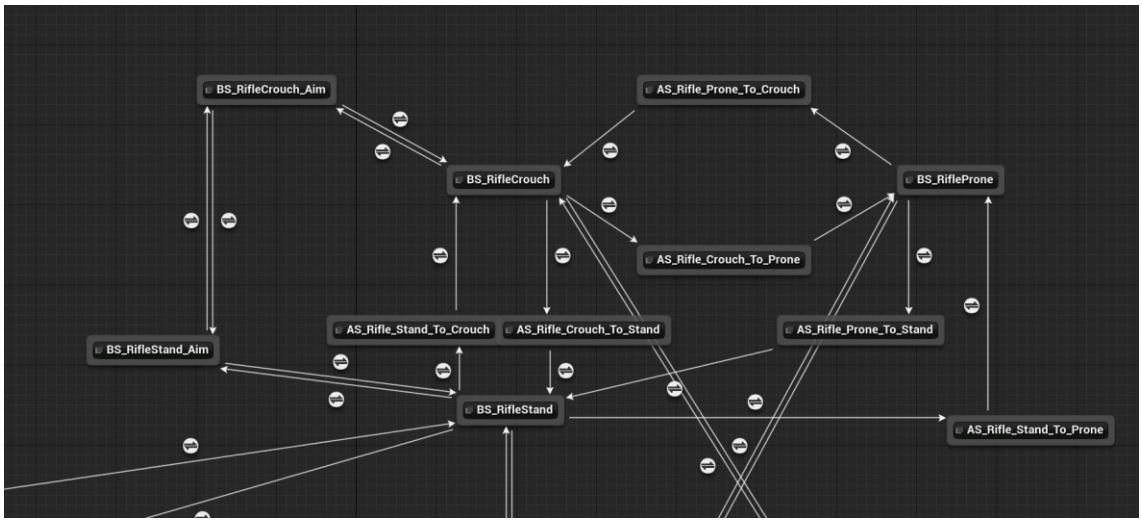


Figure 4-5 Armed state machine

The following picture shows the jump animation state machine:

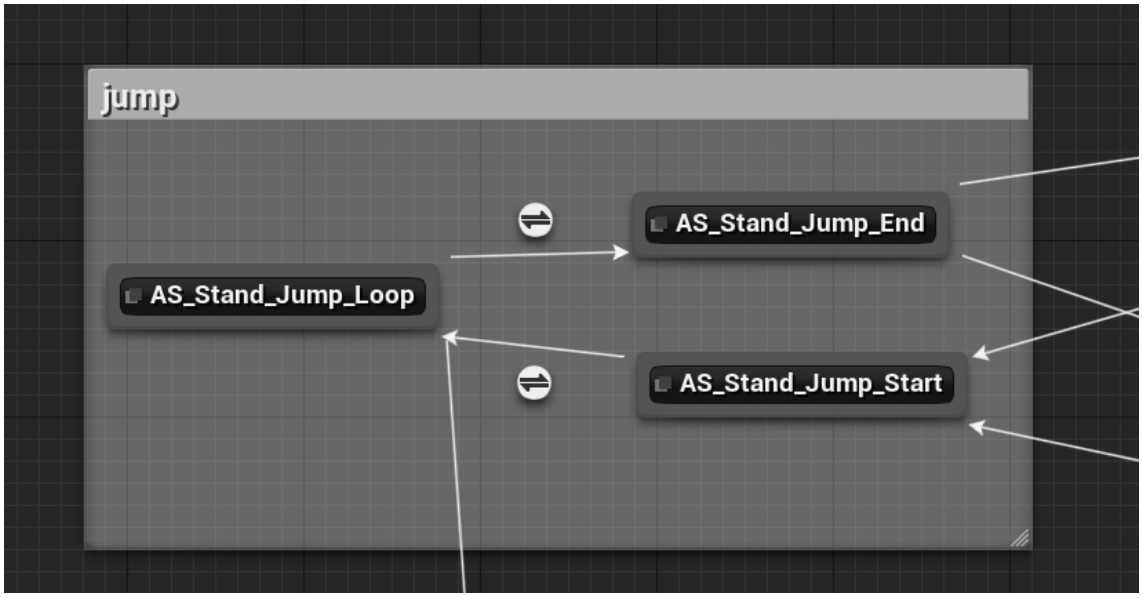


Figure 4-6 Jumping animation state machine

The picture below shows the skydiving animation state machine:

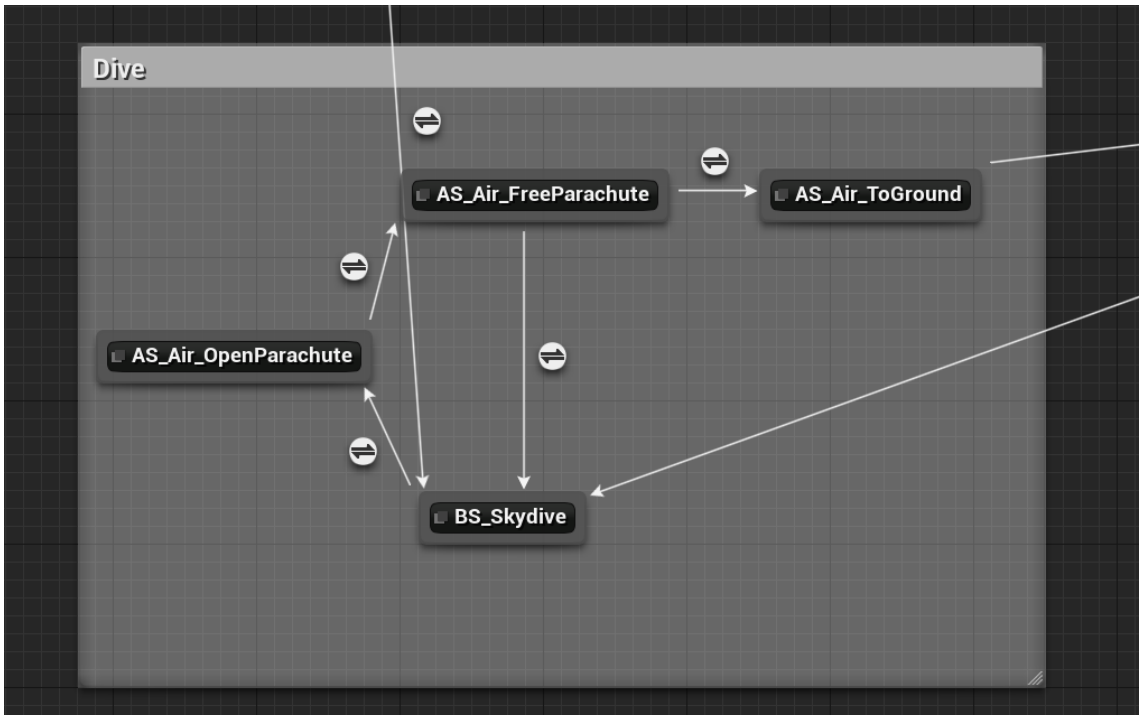


Figure 4-7 Skydiving animation state machine

Take a quick look at the table below to see what the nodes represent throughout the animation state machine.

Name	Description
BS_NormalStand	Standing Blend space when not holding a weapon
BS_NormalCrouch	Blend space in crouching position when not holding a weapon
BS_NormalProne	Prone when not holding a weapon Blend space
AS_Normal_Stand_To_Crouch	Two resources for over-animation of corresponding poses
AS_Normal_Crouch_To_Stand	Two resources for over-animation of corresponding poses
AS_Normal_Stand_To_Prone	Two resources for over-animation of corresponding poses
AS_Normal_Prone_To_Stand	Two resources for over-animation of corresponding poses
AS_Normal_Crouch_To_Prone	Two resources for over-animation of corresponding poses
AS_Normal_Prone_To_Crouch	Two resources for over-animation of corresponding poses
BS_RifleStand	Standing animation blending space when holding a weapon
BS_RifleCrouch	Crouching animation blending space when holding a weapon
BS_RifleProne	Proning animation blending space when holding a weapon

AS_Rifle_Stand_To_Crouch	Two resources for over-animation of corresponding poses
AS_Rifle_Crouch_To_Stand	Two resources for over-animation of corresponding poses
AS_Rifle_Crouch_To_Prone	Two resources for over-animation of corresponding poses
AS_Rifle_Prone_To_Crouch	Two resources for over-animation of corresponding poses
AS_Rifle_Prone_To_Stand	Two resources for over-animation of corresponding poses
AS_Rifle_Stand_To_Prone	Two resources for over-animation of corresponding poses
AS_Stand_Jump_Loop	Two resources for over-animation of corresponding poses
AS_Stand_Jump_Start	Resources for animated sequences when leaping up
AS_Stand_Jump_End	Resources for animated sequences when falling
AS_Air_OpenParachute	Resources for animated sequences when opening the umbrella
BS_Skydive	Animation blending space when landing after opening the parachute
AS_Air_FreeParachute	Resources for animation during free fall
AS_Air_ToGround	Resources for animation when falling from the air to the ground

Table 4-3

4.2 Item System

4.2.1 Item architecture

Let's start with a general overview of the architecture of the item system:

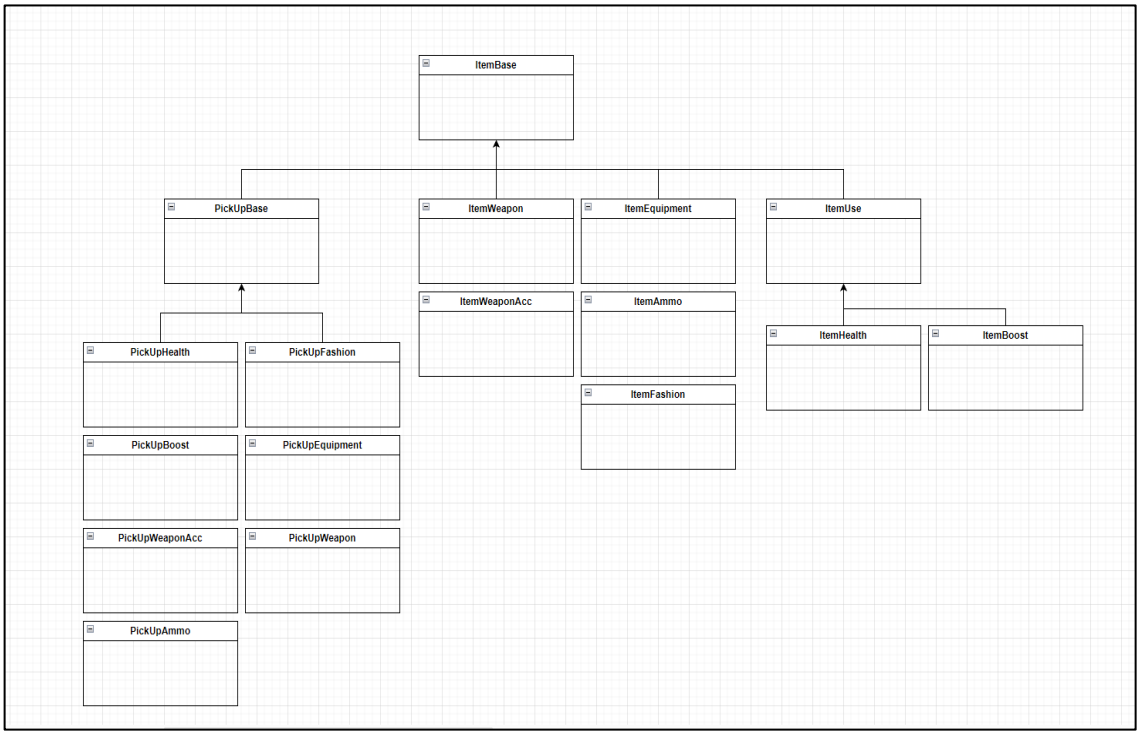


Figure 4-8 General item architecture

The following diagram shows the structure of the item base class：

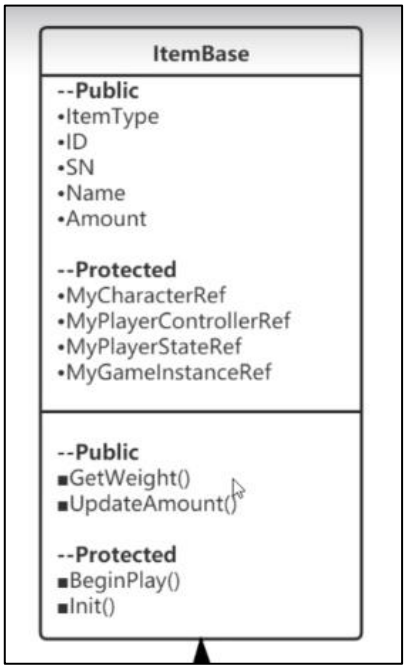


Figure 4-9 Item base class

The following diagram shows the structure of the pickup item base class：

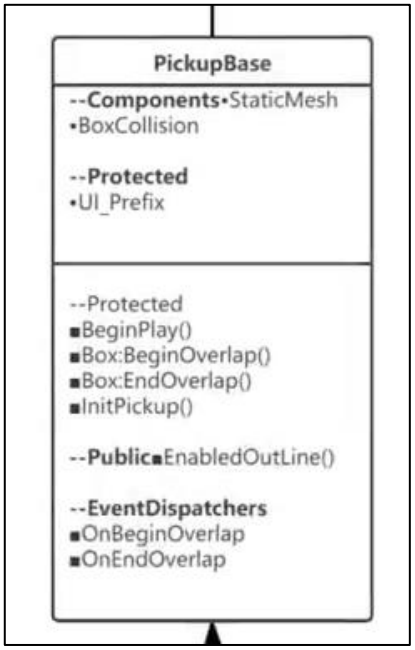


Figure 4-10 Pickup item base class

The following diagram shows the structure of the specific pickup item base class：

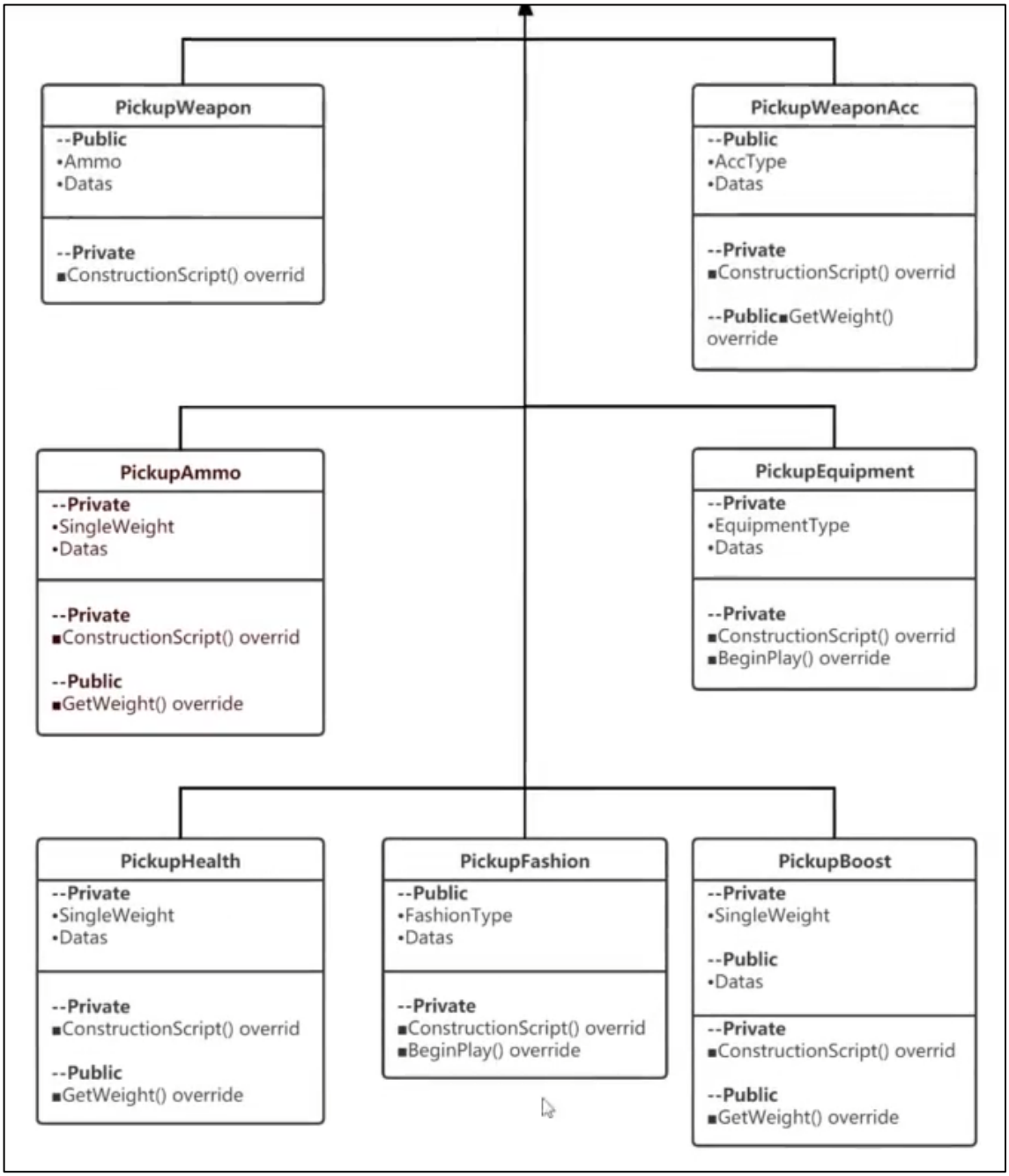


Figure 4-11 Pickup item base class

The following diagram shows the structure of the item class on the player：

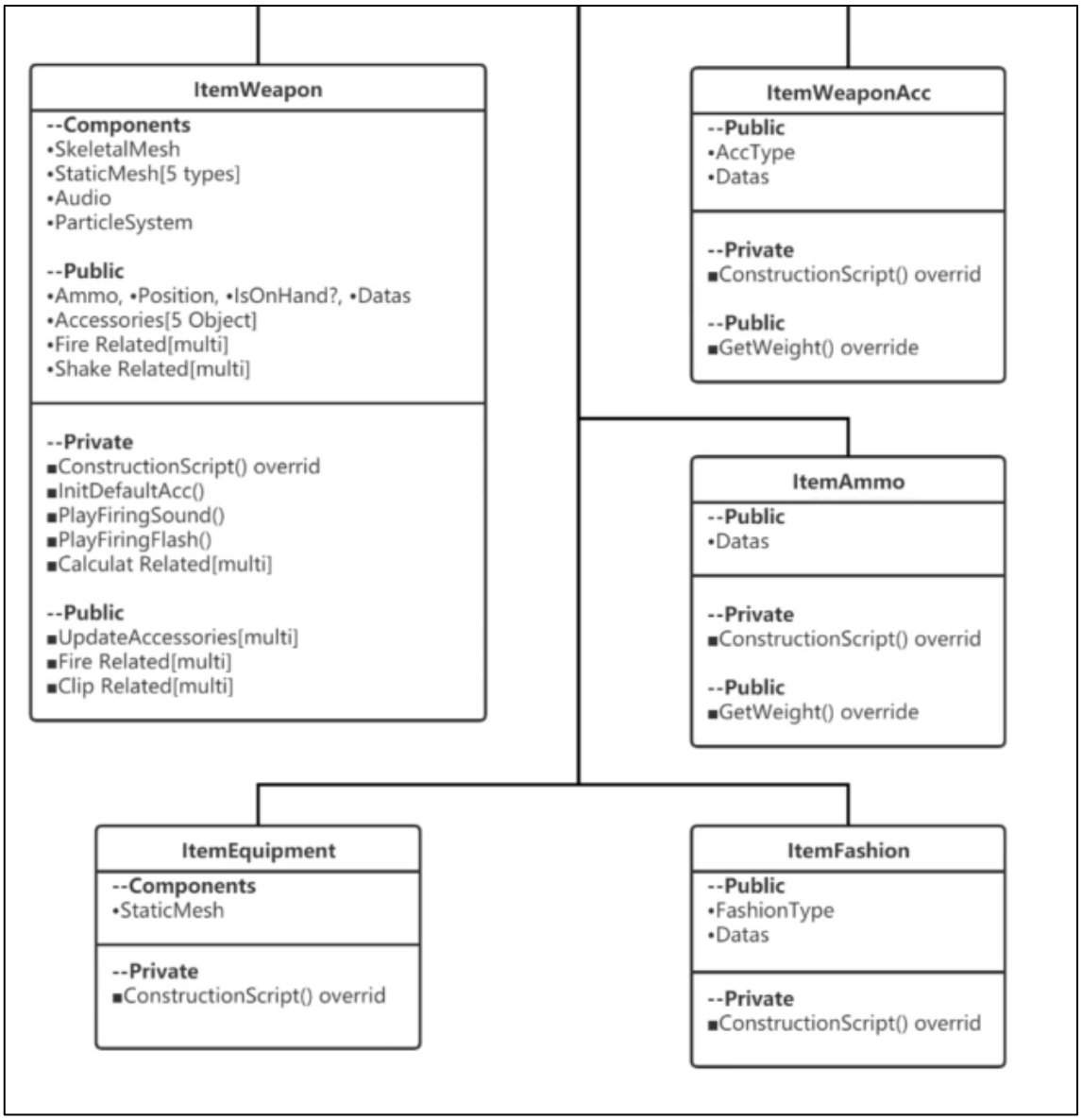


Figure 4-12 Pickup item base class

The following diagram shows the structure of the item classes that can be used：

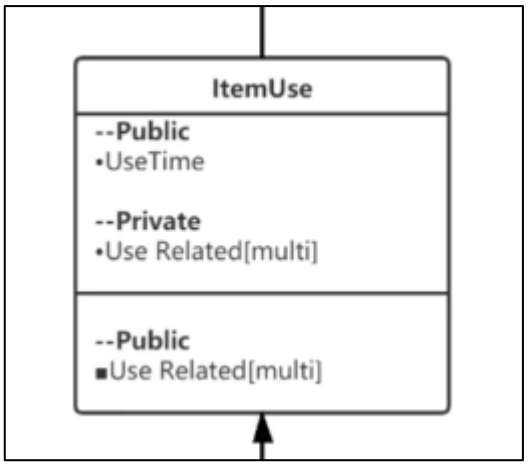


Figure 4-13 Item used class

4.2.2 Random item generation system

I provide an actor that can be dragged and dropped into the scene for random item generation similar to the AI generation system below. just drag the actor into the scene and it will generate 3-5 random items in the specified place, which will be of a random item type, and if it is a weapon type item, it will replace it with an item of a non-weapon type that is traversed to the first generated item, replacing this item with Replace the item with the bullet required for that weapon.

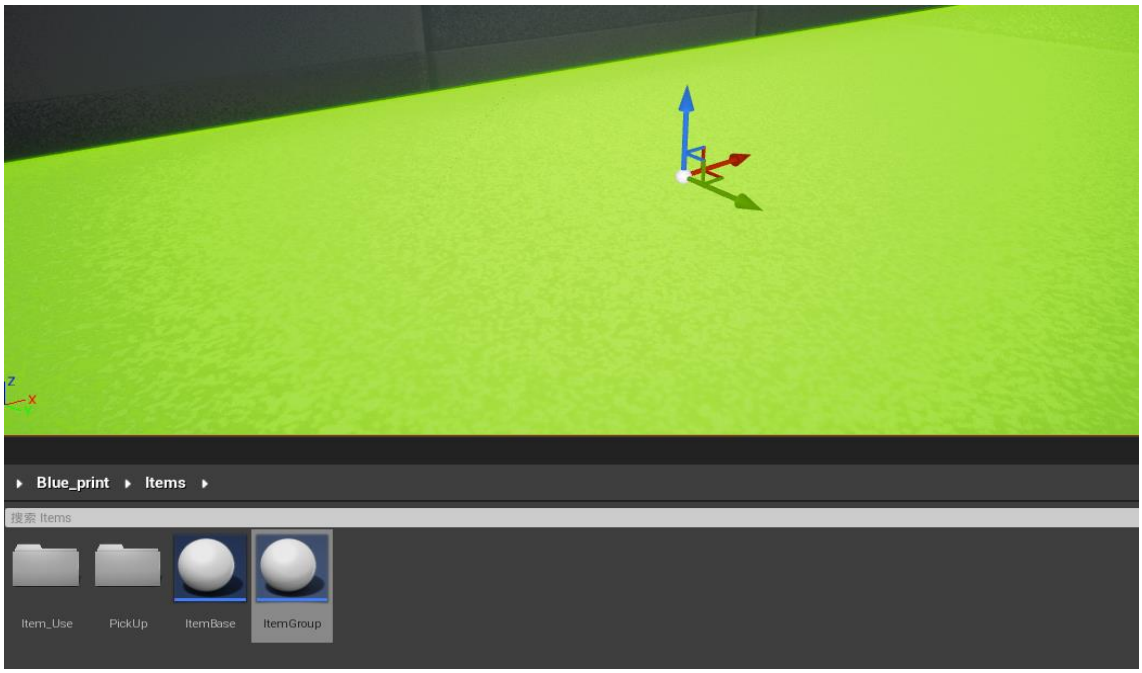


Figure 4-14 Item Group Random Generation

The following diagram shows the flow chart for generating rules:

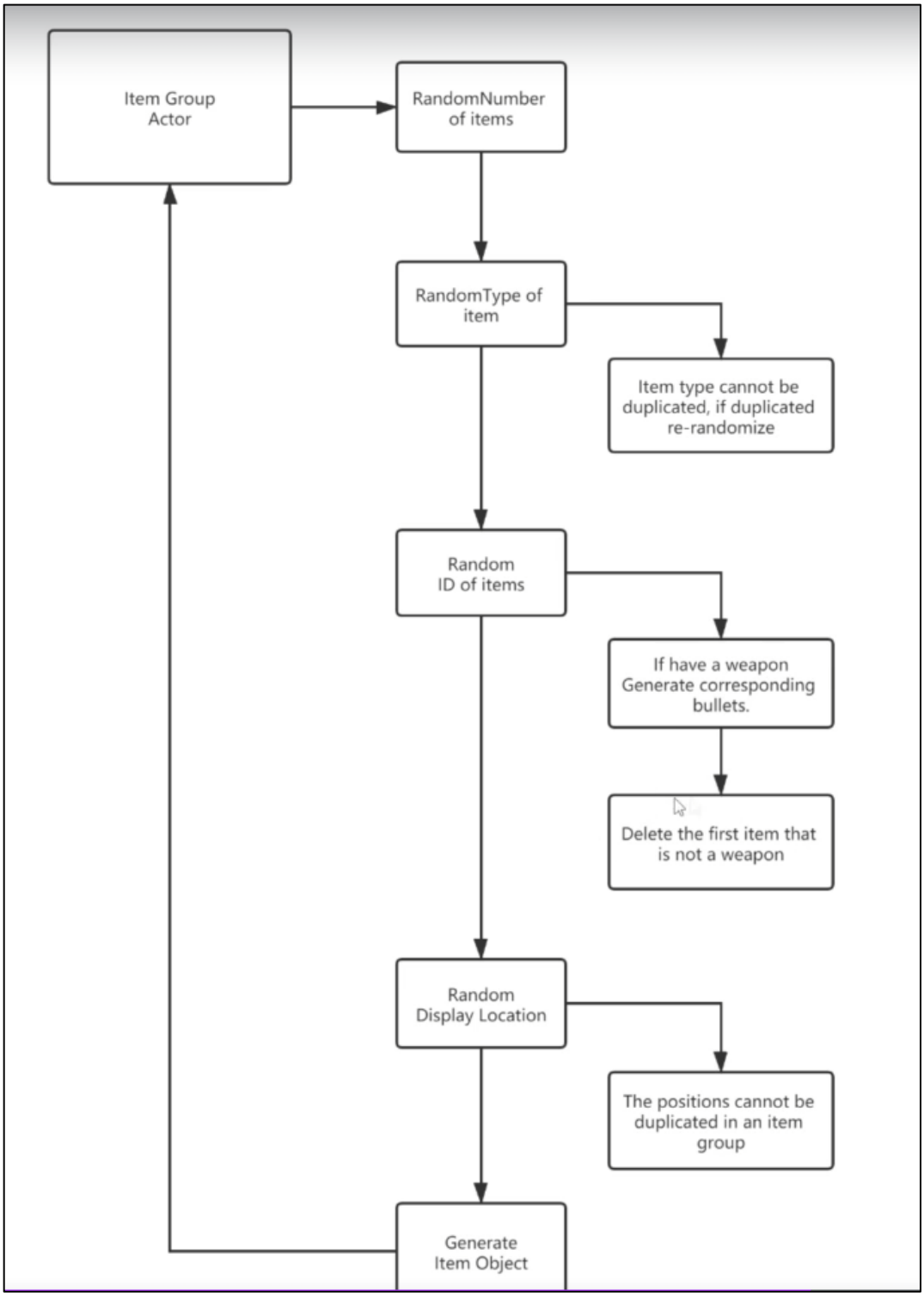


Figure 4-15 Item Group Random Generation Rules

This system rule is written specifically into the Gamemode component, and the random items on the ground are generated when you start the game.

4.3 Weapon System

This section focuses on the practical implementation of weapon systems, such as the storage of weapon data, weapon accessory systems, weapon firing and aiming functions, weapon switching functions, etc.

4.3.1 Weapon system data structure

Here you will find three tables for a quick overview of weapon data, weapon bullet data and weapon accessory data.

Name	Description
NAME	Name of weapon, e.g. M4A1
Icon	Small weapon icons
BigIcon	Weapon big icons
Thumbnail	Weapon Thumbnails
UseAmmoID	The bullet ID used by the weapon, this matches the line designation in DT_Ammo
ClipSize	Number of rounds in a magazine for this weapon
Damage	Weapon bullet damage
FiringInterval	Fire interval, the time between the firing of one round and the firing of the next round, 0 for sniper types with a single shot
ReplaceClipTime	Time taken to change magazines
ReplaceBulletTime	Time required to change bullets
AutoMode	Is it possible to switch to automatic mode, awm is not allowed

StaticMesh	Static model of the weapon
SkeletalMesh	The skeleton model of the weapon, the skeleton slots are needed to mount accessories such as magazines
DefaultMag	Default magazine for this weapon
DefaultSight	Default sight for this weapon
DefaultButtstock	Default stock for this weapon
FireSound	Sound of firing
FireSoundSuppressor	Silenced firing sound
CanSight?	Is it possible to mount a scope
CanMuzzle?	With or without muzzleloader
CanClip?	Can magazines be fitted
CanGrip?	Can be fitted with a grip
CanButtstock?	Can the stock be fitted
CrossOffsetMin_Stillness	Minimum aiming deflection when firing with the weapon stationary and without the scope on
CrossOffsetMax_Stillness	Maximum aiming deflection when firing with the weapon stationary and without the scope on
CrossOffsetMin_Move	Minimum aiming deflection when firing with the weapon in motion and without the scope on
CrossOffsetMax_Move	Maximum aiming deflection when the weapon is in motion and the scope is not on
CrossOffsetMin_AimStillness	Minimum aiming deflection when the weapon is stationary and firing with the scope on

CrossOffsetMax_AimStillness	Maximum aiming deflection when the weapon is stationary and firing with the scope on
CrossOffsetMin_AimMove	Minimum aiming deflection when the weapon is in motion and the scope is on
CrossOffsetMax_AimMove	Maximum aiming deflection when the weapon is in motion and the scope is on
VerticalOffset	Open fire vertical jitter offset value
HorizontalOffset	Firing level jitter offset value
InitVelocity	Initial velocity of a weapon bullet shot
100MVelocity	Weapon bullet velocity at 100M
300MVelocity	Velocity of weapon bullets at 300M
500MVelocity	Velocity of weapon bullets at 500M
WalkSpeedPer	
GenProbability	The weapon has a random generation probability

Table 4-4 Table Weapon Data Structure

Name	Description
Name	Bullet name
PickUpMesh	Static model with bullets on the ground
PickUpAmmo	Number of bullets at generation
Icon	Bullet icon
Weight	The capacity of a round, which affects the capacity of the pack

GenProbability	Probability of generating this bullet

Table 4-5 Weapon ammo Data Structure

Name	Description
Description	Weapon accessory names
Type	Accessory type
WeaponIDs	Weapon numbers available for this accessory, 1 for M4, 2 for AK47, 3 for AWM
Weight	Backpack capacity for accessories
StaticMesh	Model of this accessory
StaticMesh2	Model 2 of this accessory
Icon	Accessories icon
SightMultiple	Magnification, 0 for this accessory is not a magnification
IsSuppressor?	Is it possible to muffle the sound?
IsNotFlash?	Is it possible to de-flame?
ClipRate	Percentage of unit price changeover time, multiplied by the default changeover time as the final changeover time
ClipCapacity	Magazine capacity
VerticalAdjPer	Vertical offset percentage, the default vertical offset multiplied by this percentage
HorizontalAdjPer	Horizontal offset percentage, the default horizontal offset multiplied by this percentage

SocketName	The name of the slot for the weapon accessory, used to indicate the specific slot location to be fitted to the weapon

Table 4-6 Weapon accessory Data Structure

4.3.2 Weapon functionality system implementation

1. Weapon Firing:
   a. we press the left button to trigger the pressfire event, open the door of the Gate node in a sequence, and then go to exit to fire.
   b. we then determine whether the firing mode is single shot or burst, burst is a delayed seconds loop to the beginning, but it would be more logical to adjust it to the end here.
   c. then execute the firing logic: play the sound, play the flash, and reduce the number of bullets.
   d. Finally the firing animation is played, and in the case of a burst, we complete the above actions within a delay time.
   e. Delay to the beginning and re-execute.
   f. or release, close the door.
2. Weapon Reload:
   There are a number of factors that can affect the bullet change action throughout the game, such as the possibility of the player cancelling the change of order while the bullet change animation is being executed.

The diagram below illustrates in detail the relevant roles of the different components and the workflow when performing the bullet change function.

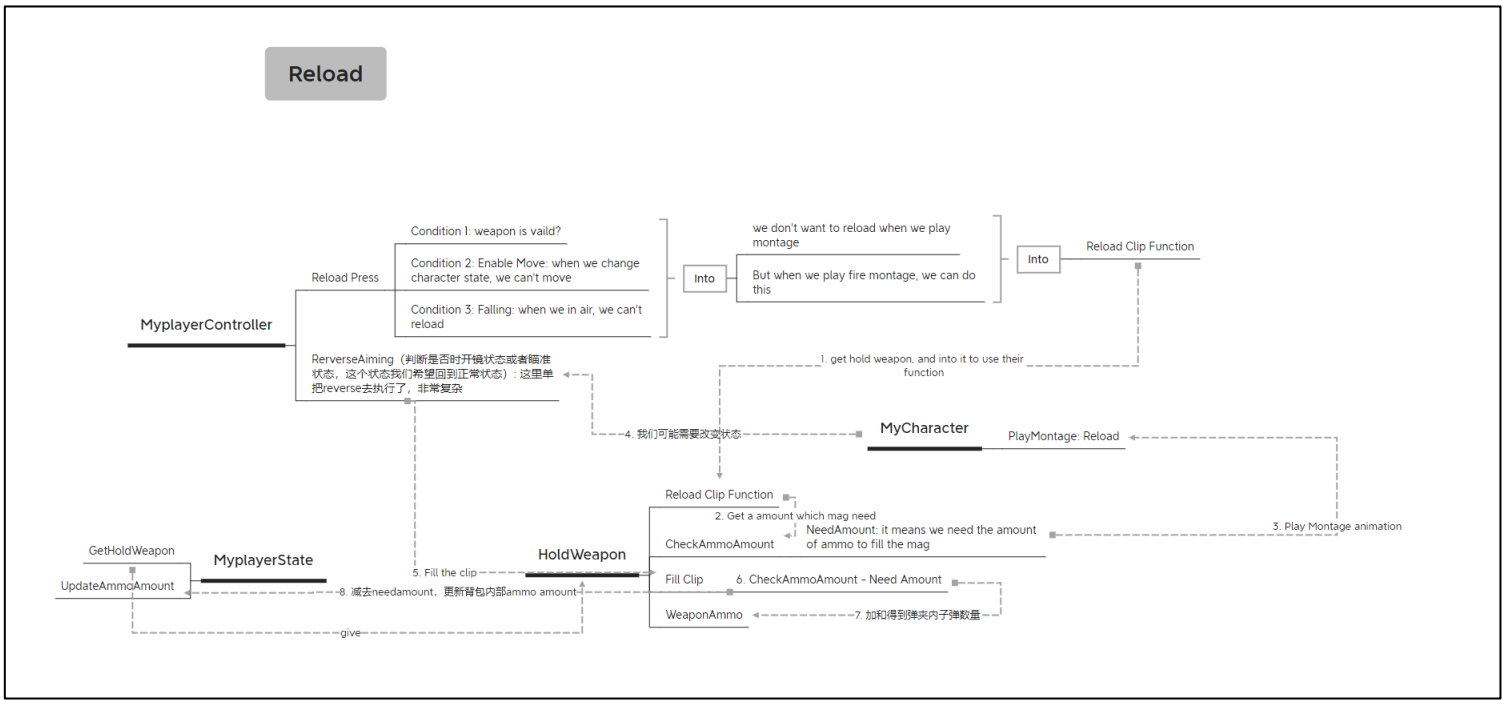


Figure 4-16 Item Group Random Generation Rules

3. Weapon aiming:

   Players can aim their weapons with open-camera precision and random fire. Smoothly articulated animations when aiming are essential to ensure a linear experience for the player.

   Precision aiming with the camera on is achieved by adjusting the camera FOV. The fov is the size of the camera's field of view: you can reduce the FOV value, used when aiming at high magnification, or enlarge the FOV value, used for the visual distortion effect caused by sprinting.

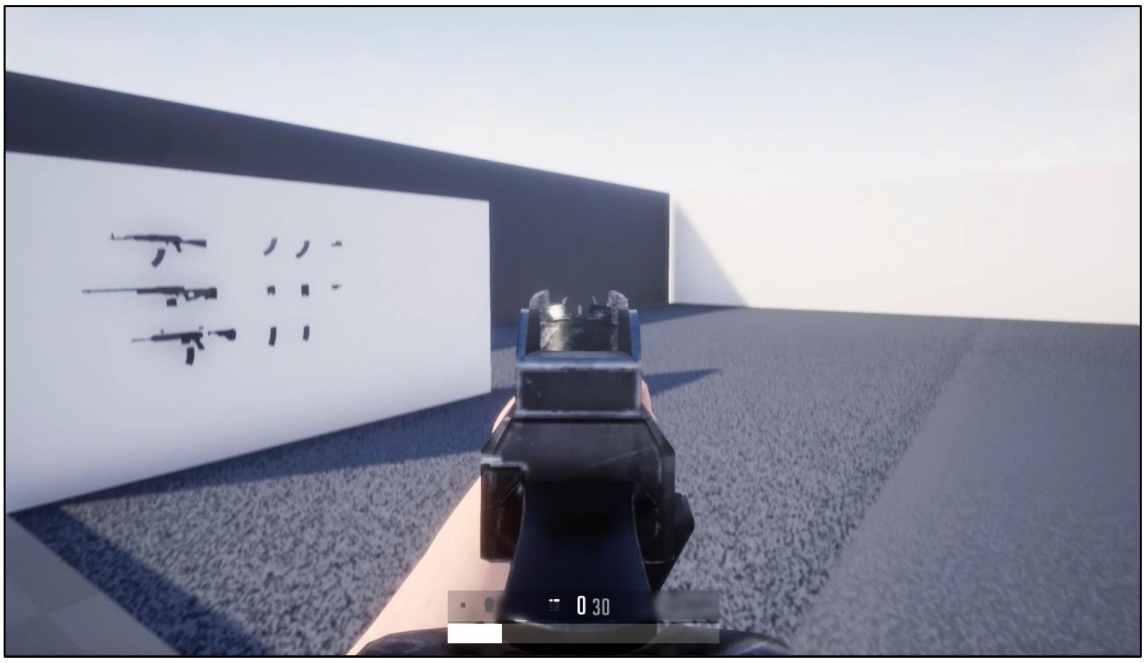


Figure 4-17 Open scope precision aiming

Figure 4-18 Random aiming

4.4 AI Spawn System

This is a placeable AI generation system. The number of AI's, their types, damage values etc. can be adjusted at any time in the detail fields, and the exposed parameters can be changed according to the needs of the game planner.

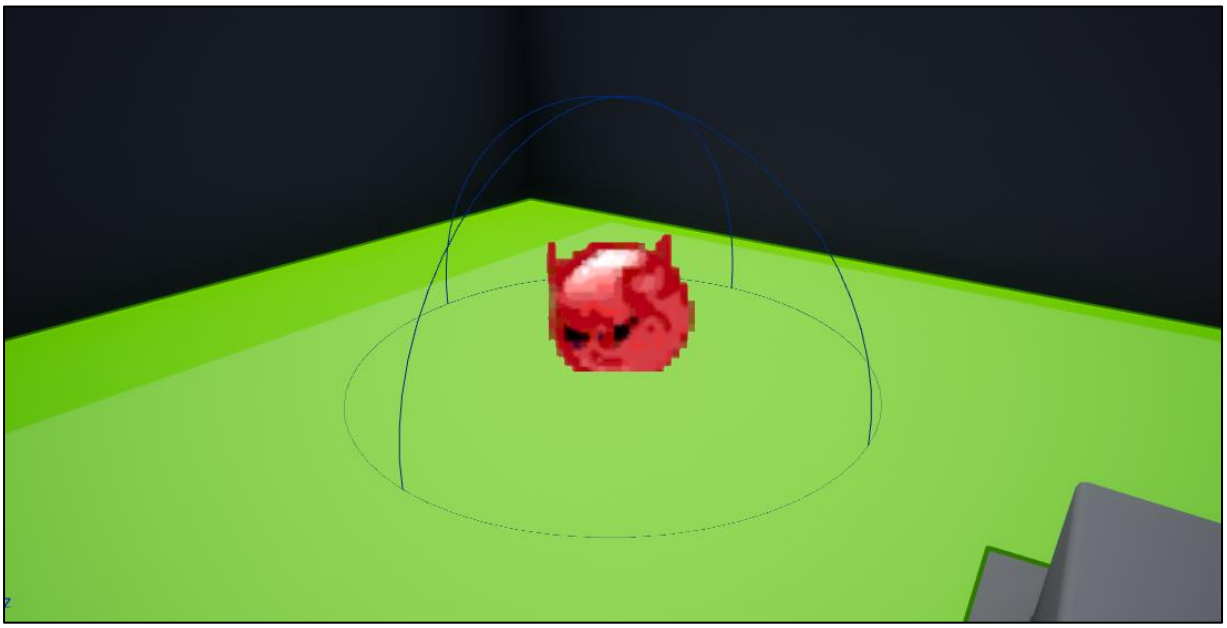


Figure 4-19 AI generation location actor

Below is a table of specific parameters that can be dynamically adjusted for AI information:

Name	Description
AI Type	Pawn type enumeration, new AI can be added at any time, by writing a new Pawn type
Number of AI generated	
Is it generated by a random number	
Is it randomly positioned to generate	

Generate location information	Specify a location or a route

Table 4-7

The following diagram shows how this AI generation system works:

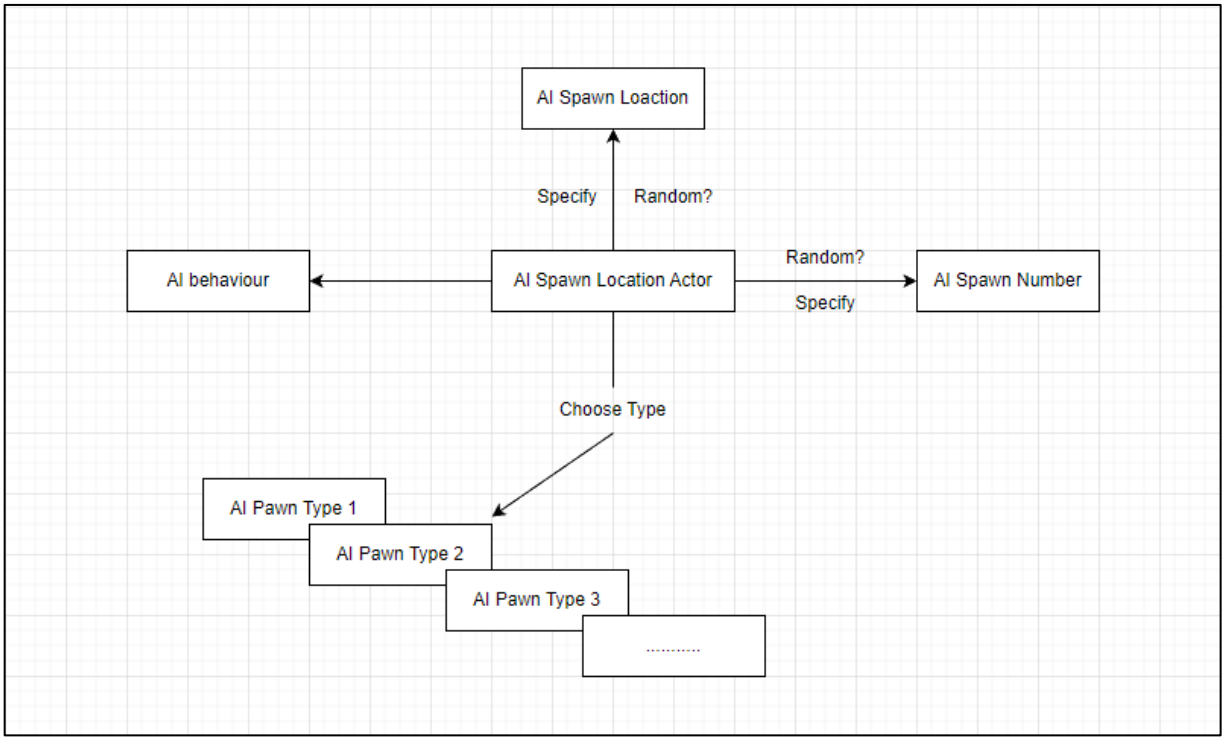


Figure 4-19 AI Spawn System workflow

Chapter 5 Conclusion

5.1 Conclusion

This paper investigates the development of shooter games based on the Unreal Engine, a powerful and easy-to-use game engine that provides a wealth of tools and resources for the development of shooter games. Through in-depth research and practical application of Unreal Engine, this paper attempts to create a complete shooter game as far as possible, and achieves good results in a real-world project.

Although I didn't end up with a flowing game, it was a good start for my introduction to game making. I tried to complete as many subsystems of the shooter as I could and it was fun to try to recreate the games I had played.

There were many challenges and difficulties during the development of the game, but I really enjoyed the process of making the game. This project has enhanced my game development skills and given me a general understanding of the techniques involved in developing a shooter game and the challenges I may encounter, which will be invaluable experience in future work on real projects.

5.2 Future work

Despite the achievements of this paper in the development of games in the shooter genre, there are still many areas that could be improved and studied in depth. Future work will focus on the following directions:

1. Enhancing game interaction: In terms of game design, we will further investigate how players interact with the game world in order to improve the immersion and fun of the game. For example, more rich mission systems can be developed, tactical and strategic elements introduced, and the skills and equipment of the game characters increased.
2. Add multiplayer online game functionality: For online multiplayer games, I will try to delve into techniques for network synchronisation, latency

optimisation and security protection to improve the stability and gaming experience of multiplayer games.

3. Social features and eSports: We will consider adding more social features to the game, such as a buddy system, leaderboards and chat features, to increase interaction between players. In addition, we will also look at ways to combine the shooter genre with eSports to explore its potential in the professional competitive arena.
4. Improving the personal warehouse system: The project has yet to build a personal warehouse system that can be used to store scarce resources and supplies that players collect on the map. A decent personal storage system will be created in the future to give players a better desire to collect.

References

[1] Mallinson, A. (2017). The history of the first person shooter. PC Gamer.

[2] Leigh, R. (2014). The Making of Half-Life. In R. K. Pooley & T. Marrinan (Eds.), You Died: The Dark Souls Companion (pp. 165-173).

[3] Lombardi, D. (2000). Valve Acquires Counter-Strike. Valve Corporation.

[4] Newzoo. (2021). Global Games Market Report 2021. Newzoo.

[5] Jenny, S. E., Manning, R. D., Keiper, M. C., & Olrich, T. W. (2017). Virtual(ly) athletes: where eSports fit within the definition of 'sport'. Quest, 69(1), 1-18.

[6] Lee, D., & Schoenstedt, L. J. (2011). Comparison of eSports and traditional sports consumption motives. ICHPER-SD Journal of Research, 6(2), 39-44.

[7] Voida, A., Carpendale, S., & Greenberg, S. (2010). The individual and the group in console gaming. In Proceedings of the 2010 ACM conference on Computer Supported Cooperative Work (pp. 371-380).

[8] Kowert, R., Domahidi, E., & Quandt, T. (2014). The relationship between online video game involvement and gaming-related friendships among emotionally sensitive individuals. Cyberpsychology, Behavior, and Social Networking, 17(7), 447-453.

[9] Leino, O. T. (2018). The cybernetics of shooting: Computer games, guns and technological imagination. Convergence, 24(4), 345-362.

[10] Cummings, J. J., & Bailenson, J. N. (2016). How immersive is enough? A meta-analysis of the effect of immersive technology on user presence. Media Psychology, 19(2), 272-309.

[11] 占绍文, 赵宇晴. 移动游戏产品更要突出社会效益. 中国出版, 2018(19): 58-61

[12] Unreal Engine. (n.d.). Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/

[13] Epic Games. (n.d.-a). Game State. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/API/Runtime/Engine/GameFramework/AGameStateBase/

[14] Epic Games. (n.d.-b). Game Instance. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/API/Runtime/Engine/Engine/UGameInstance/

[15] Epic Games. (n.d.-c). HUD. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/API/Runtime/Engine/GameFramework/AHUD/

[16] Epic Games. (n.d.-d). Player State. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/API/Runtime/Engine/GameFramework/APlayerState/

[17] Epic Games. (n.d.-e). Blueprint Performance. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/ProgrammingAndScripting/Blueprints/Performance/

[18] Epic Games. (n.d.-f). C++ Programming Tutorials. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/ProgrammingAndScripting/ProgrammingWithCPP/CPPTutorials/

[19] Epic Games. (n.d.-g). Animation Blueprints. Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/AnimatingObjects/AnimationBlueprints/

[20] Epic Games. (n.d.-h). Unreal Motion Graphics UI Designer (UMG). Unreal Engine 4 Documentation. Retrieved from https://docs.unrealengine.com/4.27/en-US/InteractiveExperiences/UMG/

[21] Rollings, A., & Adams, E. (2003). Andrew Rollings and Ernest Adams on game design. New Riders.

[22] Kasurinen, J. (2015). Understanding software engineering practices in video game development. Tampere University of Technology.

[23] Adams, E. (2014). Fundamentals of Game Design. New Riders.

[24] Rogers, Y., Sharp, H., & Preece, J. (2011). Interaction design: Beyond human-computer interaction. John Wiley & Sons.

[25] Bartle, R. (1996). Hearts, clubs, diamonds, spades: Players who suit MUDs. Journal of MUD Research, 1(1), 19.

[26] Ryan, R. M., Rigby, C. S., & Przybylski, A. (2006). The motivational pull of video games: A self-determination theory approach. Motivation and Emotion, 30(4), 344-360.

[27] Salen, K., & Zimmerman, E. (2003). Rules of play: Game design fundamentals. MIT Press.

[28] El-Nasr, M. S., Drachen, A., & Canossa, A. (Eds.). (2013). Game analytics: Maximizing the value of player data. Springer Science & Business Media.

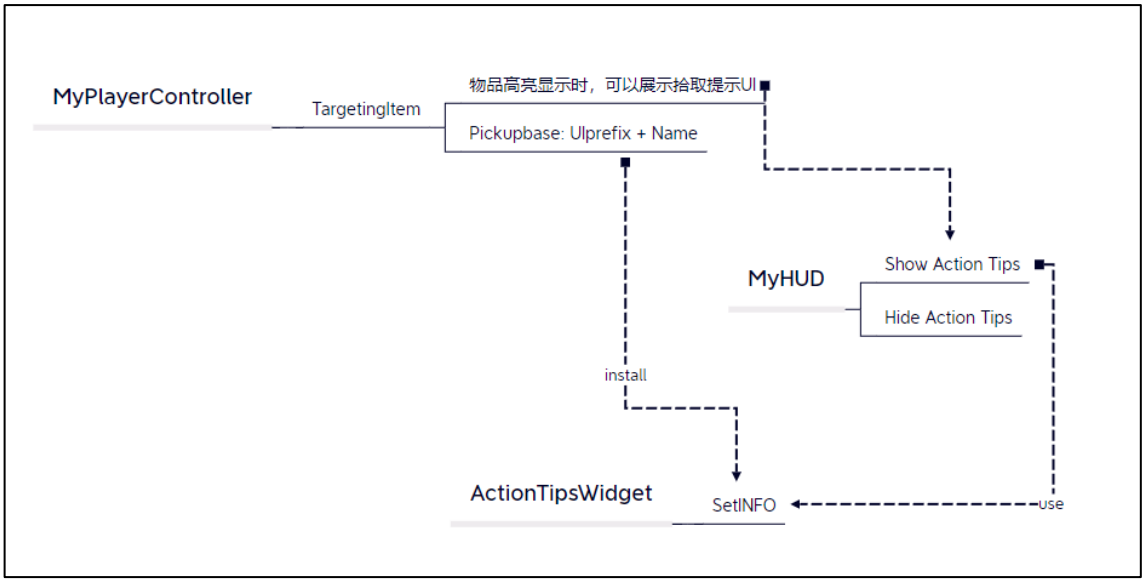
[29] Hunicke, R., LeBlanc, M., & Zubek, R. 2004. MDA: A Formal Approach to Game Design and Game Research. Proceedings of the AAAI Workshop on Challenges in Game AI, 4(1)

[30] Vella, K. (2018). The role of consumables in competitive online multiplayer games: Balancing gameplay and player experience. Entertainment Computing, 25, 60-71.

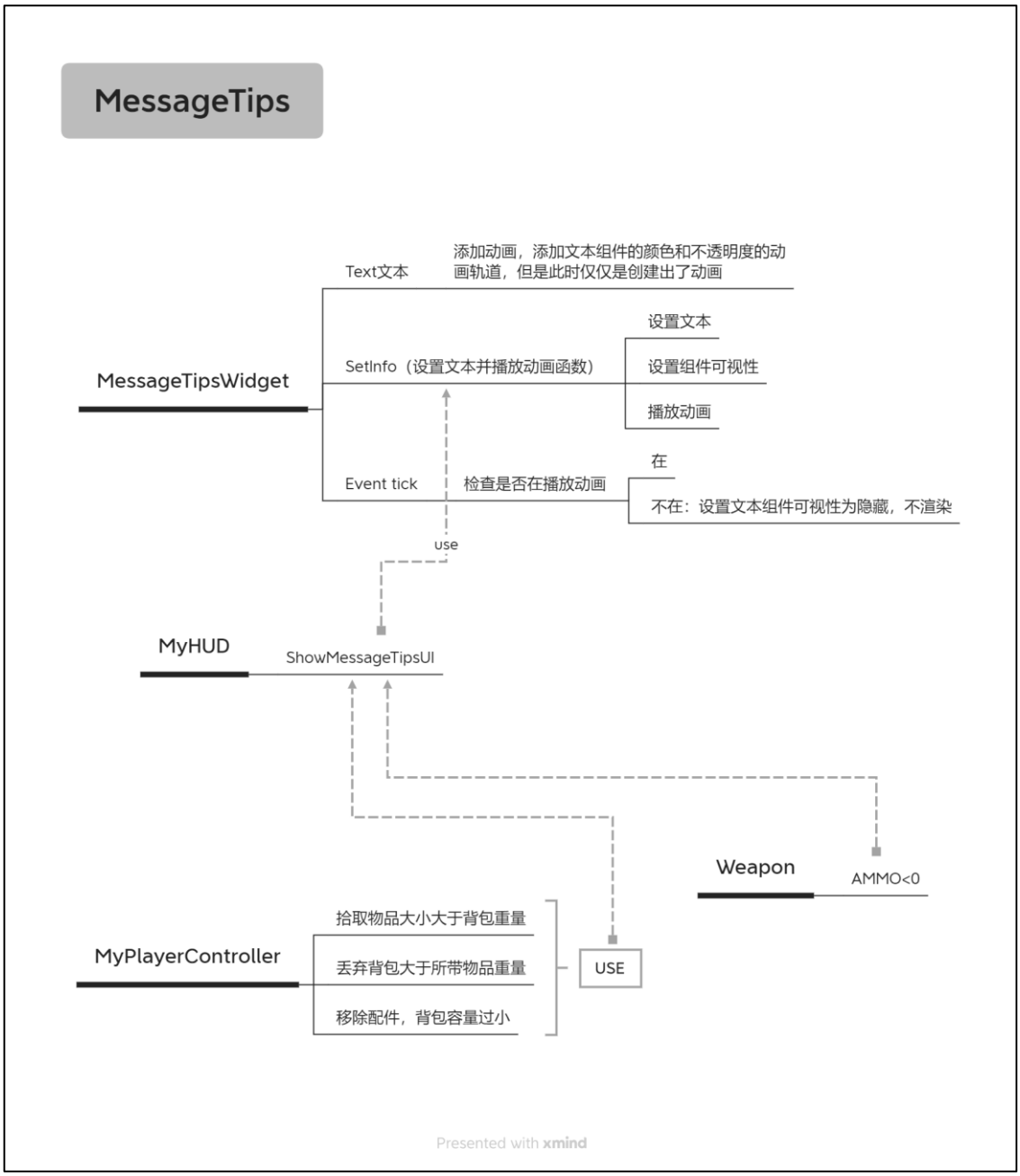
[31] Brown, J. A. (2017). Player tactics in online multiplayer shooters: The impact of consumables and power-ups. Entertainment Computing, 20, 45-54.

Appendix

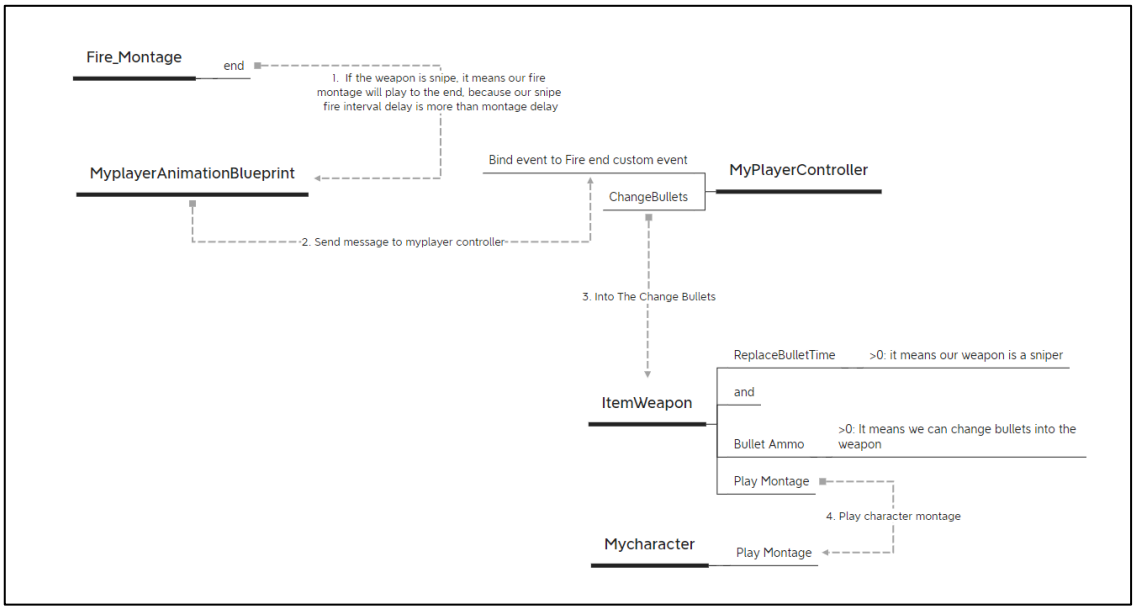
Action Tips Flowchart:



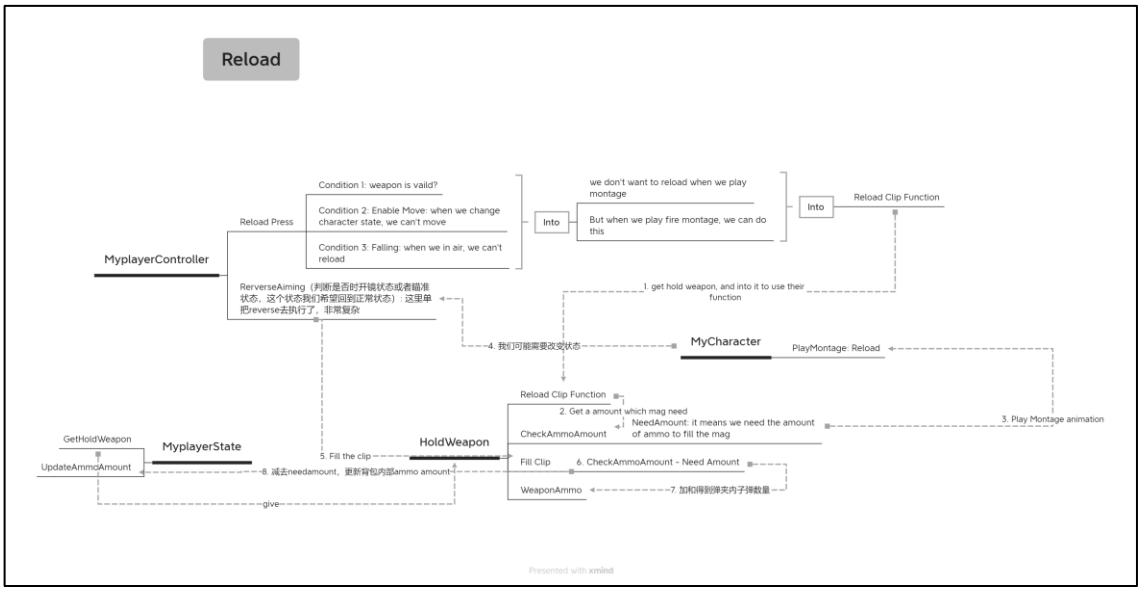
Message Tips Flowchart:



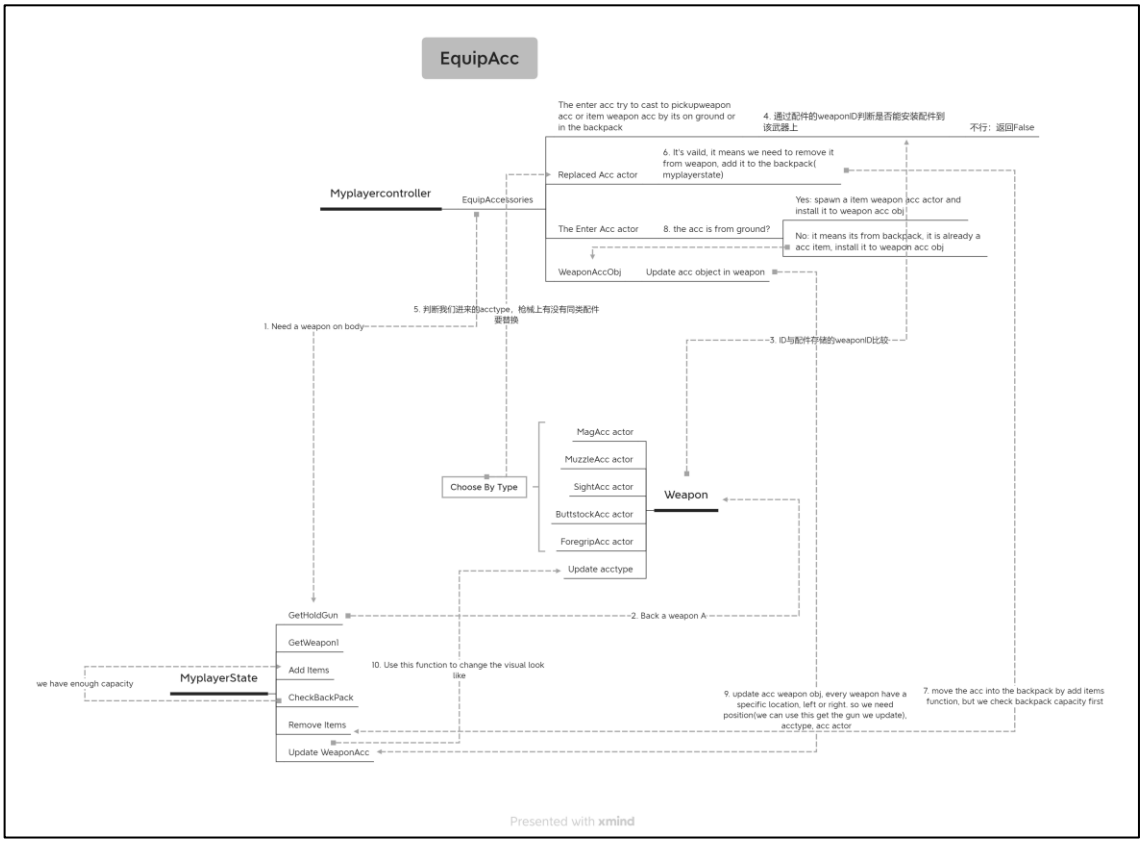
Change Bullets Flowchart:



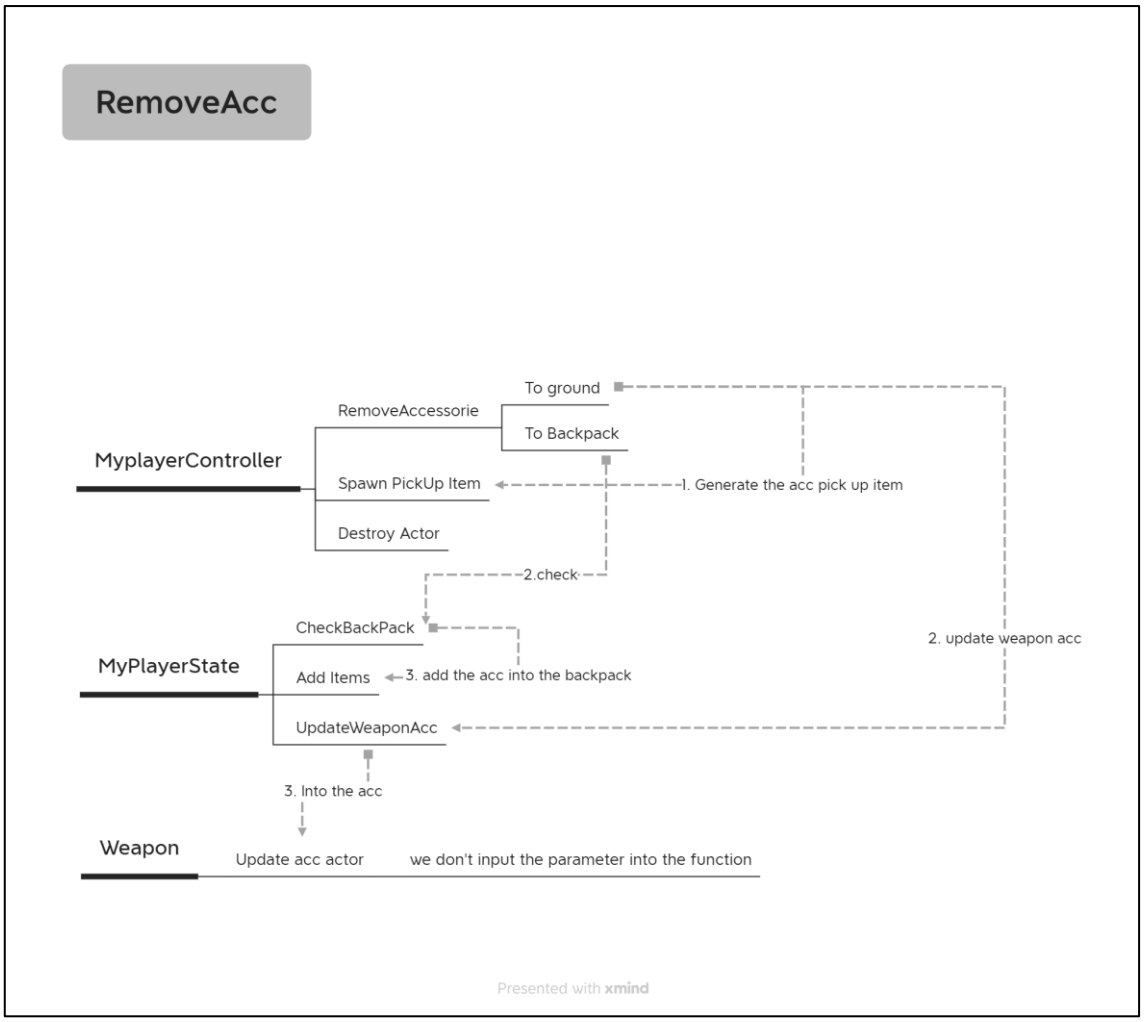
Weapon Reload Flowchart:



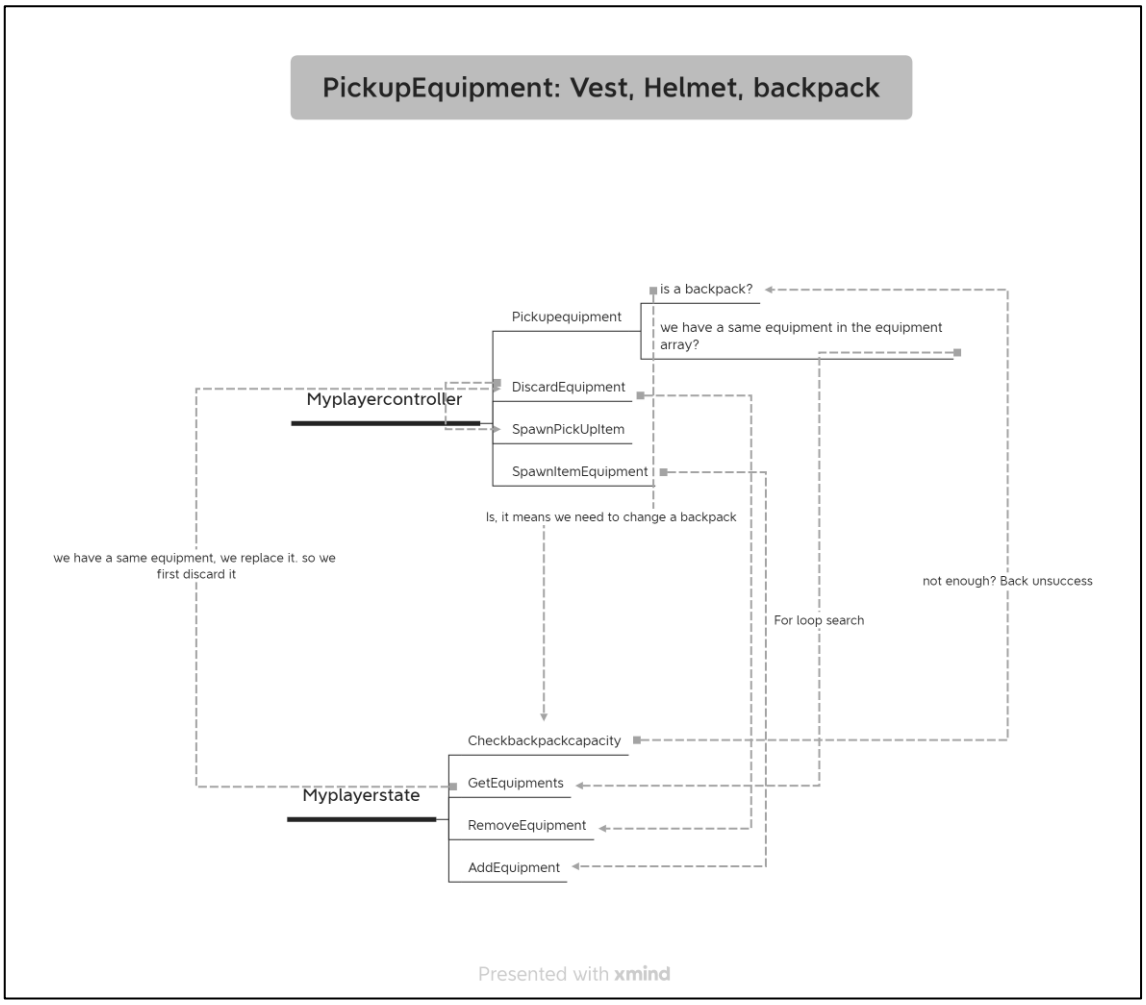
Equip weapon accessories:



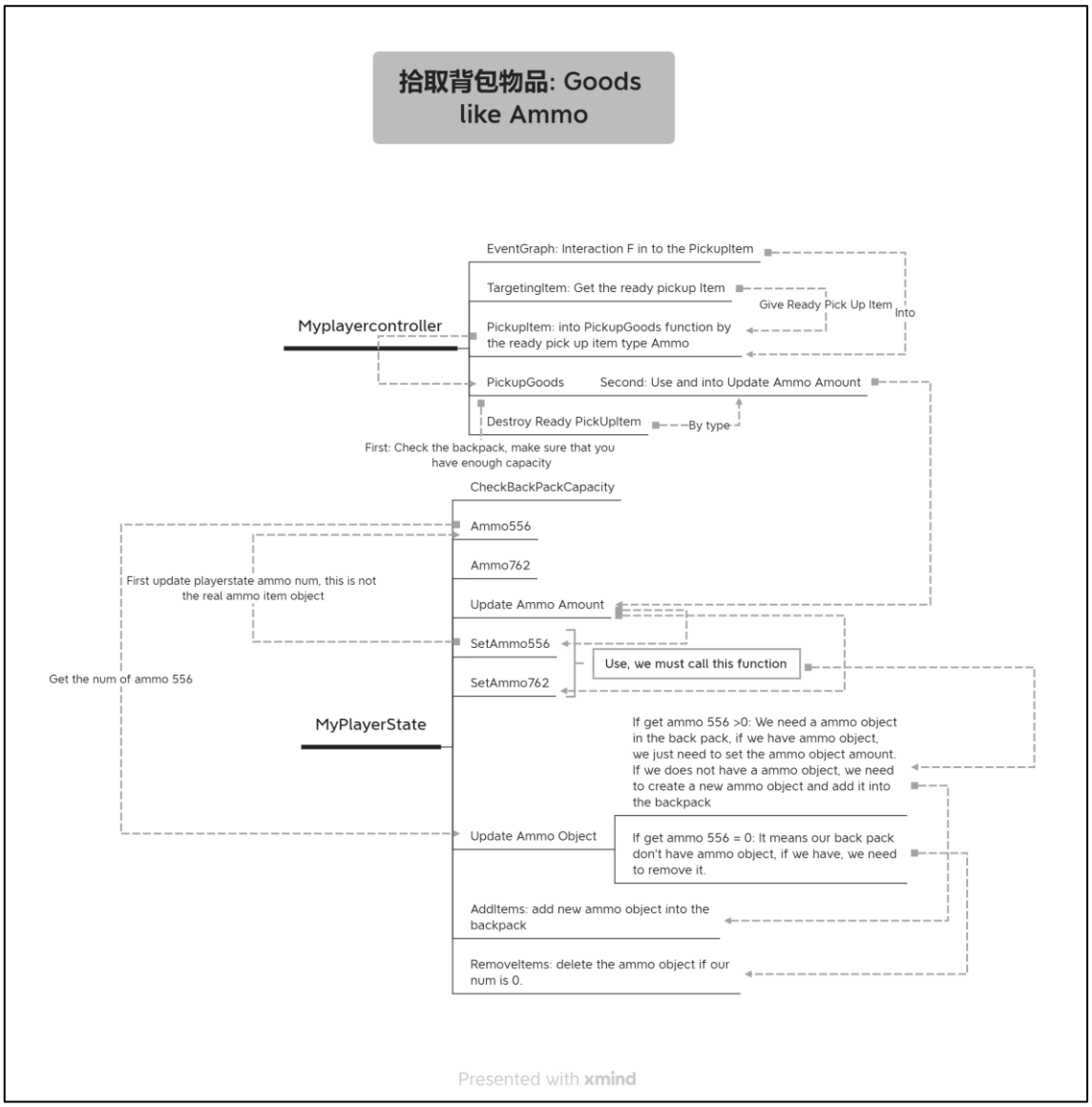
Remove weapon accessories:



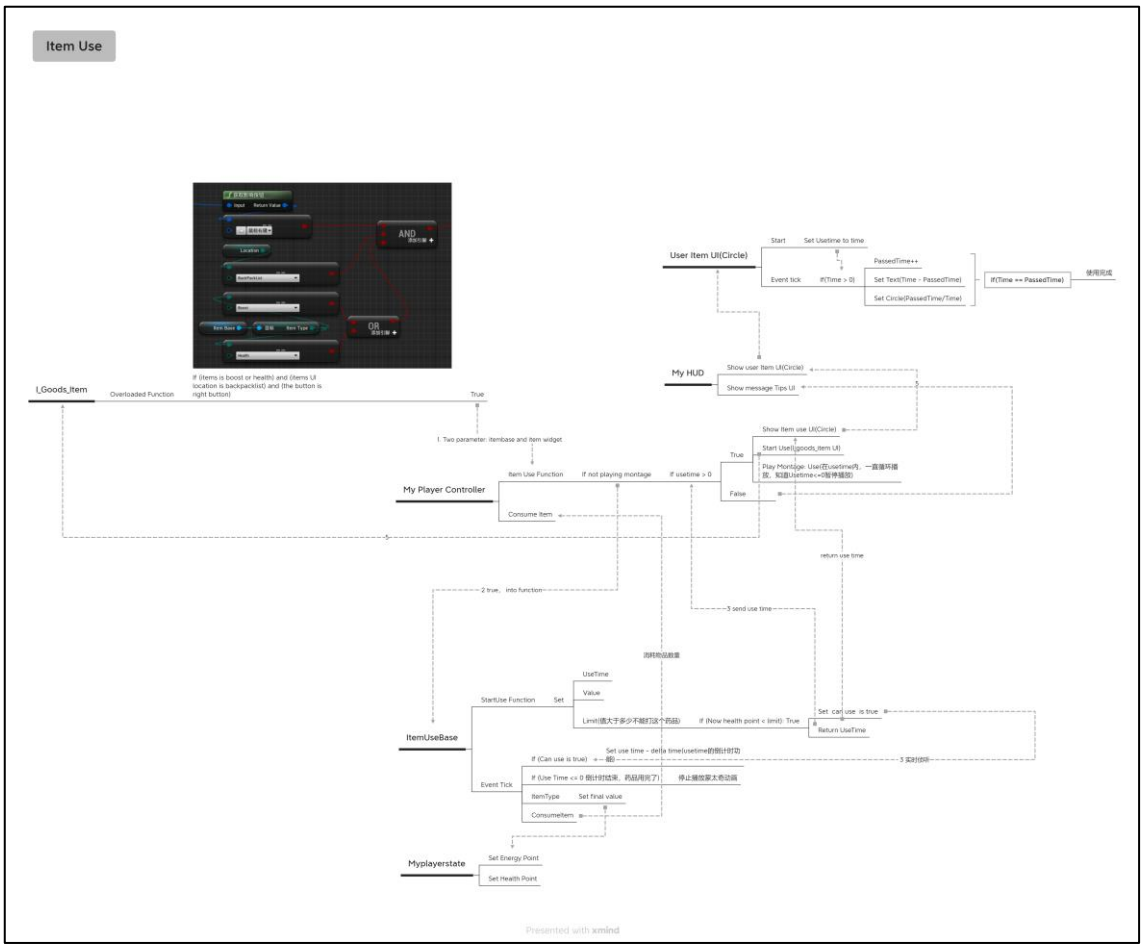
Pickup Equipment Flowchart:



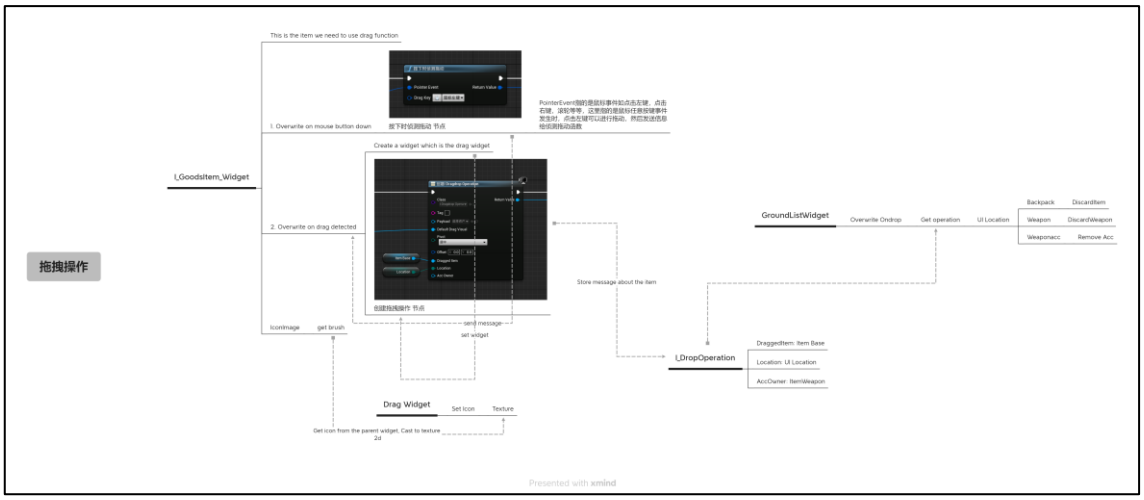
Pickup ammo flowchart:



The character use the item flowchart:



Interface interaction flow chart:



Acknowledgements

First of all, I would like to thank my supervisor (Professor WONG, Hon-Cheng) for the support and understanding he has given me throughout the writing process of this project.

Secondly, I would also like to thank my classmates and housemates, although we have different aspirations, we all have our own things that we want to do and enjoy, and I have learnt a lot from them. I have learnt a lot from them. Especially during the production process, my classmate (Jiawen Luo) has given me a lot of substantial support.